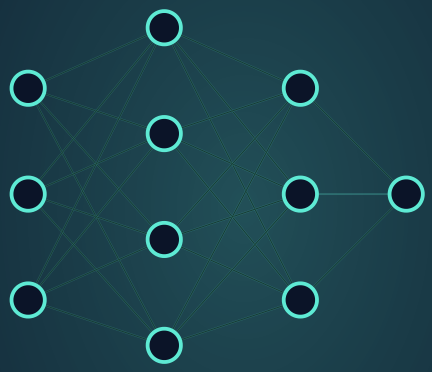


VEILLE TECHNOLOGIQUE · DEEP LEARNING

Vision, séquences & attention

Réseaux convolutifs, réseaux récurrents et Transformers : théorie, mathématiques, illustrations et mise en pratique avec PyTorch & Hugging Face.

09

Réseaux convolutifs & apprentissage par transfert

Convolution, pooling, CNN, ResNet, modèles préentraînés, fine-tuning

10

Données séquentielles & réseaux récurrents

Séries temporelles, fenêtres, RNN, BPTT, LSTM, GRU, validation chronologique

11

Attention & Transformers

Tokenisation, Q/K/V, multi-têtes, positions, BERT/GPT/T5, Hugging Face

Sommaire

Veille Deep Learning — Parties 9, 10 et 11 : vision, séquences et attention.

Partie 9 · Réseaux convolutifs et apprentissage par transfert 2

9.1 Représentation numérique des images 2
9.2 Convolution, filtres et partage des poids 4
9.3 Canaux, stride, padding et pooling 5
9.4 Construction et entraînement d'un CNN 7
9.5 Comparaison entre MLP et CNN 10
9.6 Utilisation d'un modèle préentraîné 10
9.7 Extraction de représentations et fine-tuning 11

Partie 10 · Données séquentielles et réseaux récurrents 15

10.1 Séquences et séries temporelles 15
10.2 Fenêtres temporelles et préparation des données 16
10.3 Réseaux récurrents (RNN) 18
10.4 État caché et dépendances temporelles 19
10.5 Introduction aux LSTM et GRU 20
10.6 Évaluation avec un découpage chronologique 22

Partie 11 · Attention et Transformers 24

11.1 Tokenisation et embeddings 24
11.2 Mécanisme d'attention et auto-attention 25
11.3 Query, key et value 27
11.4 Attention à plusieurs têtes 28
11.5 Information de position 29
11.6 Architecture générale des Transformers 30
11.7 Encodeurs et décodeurs 31
11.8 Utilisation et adaptation de modèles préentraînés avec Hugging Face 32

Annexe · Formulaire et références 37

A Formulaire de synthèse 37
B Références essentielles 37

PARTIE 9

09

Réseaux convolutifs et apprentissage par transfert

Les réseaux de neurones convolutifs (CNN) exploitent la structure spatiale des images : connexions locales, poids partagés et hiérarchie de représentations. L'apprentissage par transfert permet ensuite de réutiliser un CNN entraîné sur des millions d'images pour résoudre une nouvelle tâche avec peu de données.

9.1 Représentation numérique des images
9.2 Convolution, filtres et partage des poids
9.3 Canaux, stride, padding et pooling
9.4 Construction et entraînement d'un CNN
9.5 Comparaison entre MLP et CNN
9.6 Utilisation d'un modèle préentraîné
9.7 Extraction de représentations et fine-tuning

9.1 Représentation numérique des images

Pour un ordinateur, une image n'est qu'un **tableau de nombres**. Chaque case de ce tableau est un *pixel* (*picture element*) dont la valeur code une intensité lumineuse.

DÉFINITION

Une image numérique de hauteur H, de largeur W et à C canaux est un **tenseur** $X \in \mathbb{R}^{C \times H \times W}$. L'élément $X_{c,i,j}$ est l'intensité du canal c au pixel situé à la ligne i et à la colonne j.

- **Niveaux de gris** : un seul canal ($C = 1$). Valeur entière sur 8 bits, de 0 (noir) à 255 (blanc).
- **Couleur RVB (RGB)** : trois canaux ($C = 3$), rouge, vert et bleu. Chaque pixel est un triplet $(r, g, b) \in \{0, \ldots, 255\}^3$, soit $256^3 \approx 16{,}7$ millions de couleurs.
- **Autres cas** : canal alpha (transparence, $C = 4$), images médicales 3D ($C \times D \times H \times W$), images multispectrales satellite (C = 10 à 200 bandes), vidéos (une dimension temporelle en plus).

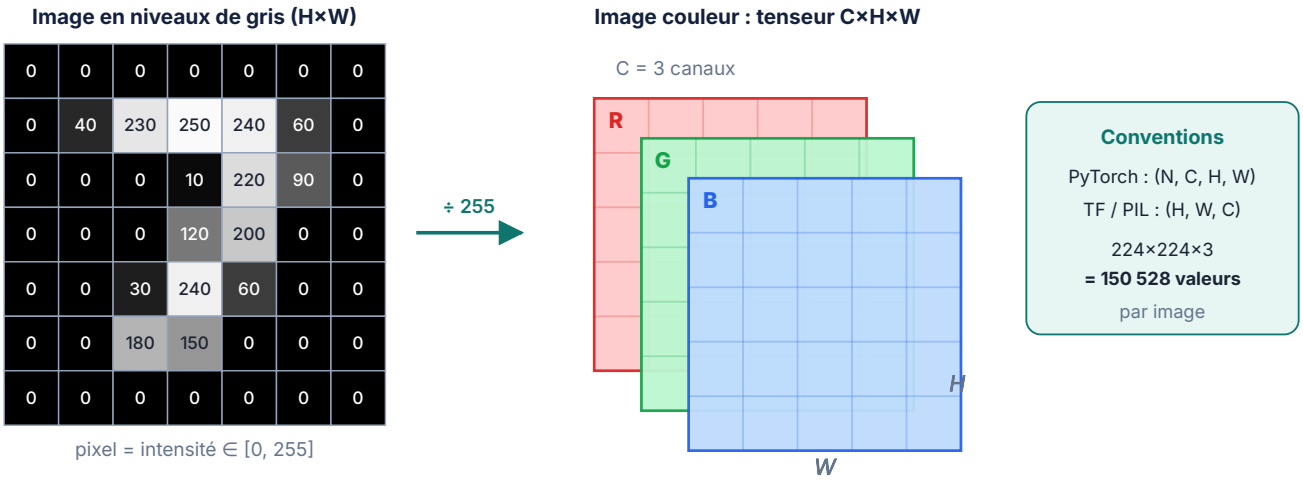


Figure 9.1 — Une image est une grille de pixels ; en couleur, on empile trois grilles (canaux R, G, B).

Conventions de dimensions

Un lot (*batch*) d'images forme un tenseur 4D. **PyTorch** utilise le format `(N, C, H, W)` (« channels first ») ; PIL, OpenCV, Matplotlib et TensorFlow utilisent `(H, W, C)` (« channels last »). Une conversion est donc souvent nécessaire : `x.permute(2, 0, 1)`. Attention : OpenCV lit les images en **BGR** et non en RGB.

Prétraitement standard

1. **Redimensionnement** à une taille fixe (ex. 224×224), car un lot doit contenir des tenseurs de même forme.
2. **Mise à l'échelle** vers $[0, 1]$: $x \leftarrow x/255$ (fait automatiquement par `transforms.ToTensor()`).
3. **Standardisation par canal** avec la moyenne μ_c et l'écart-type σ_c calculés sur le jeu d'entraînement :

$$\tilde{x}_{c,i,j} = \frac{x_{c,i,j} - \mu_c}{\sigma_c}, \qquad \text{ImageNet} : \mu = (0{,}485;\ 0{,}456;\ 0{,}406),\ \sigma = (0{,}229;\ 0{,}224;\ 0{,}225).$$

POURQUOI LES IMAGES POSENT PROBLÈME À UN MLP

Une image $224 \times 224 \times 3$ contient $150\,528$ valeurs. Une couche dense vers $1\,000$ neurones nécessiterait déjà ≈ 150 millions de poids. De plus, aplatir l'image détruit la notion de voisinage : deux pixels voisins deviennent deux entrées sans relation particulière. Les CNN résolvent ces deux problèmes.

```python
from PIL import Image
import torchvision.transforms as T

img = Image.open("chat.jpg").convert("RGB")    # (H, W, 3), valeurs 0..255
transform = T.Compose([
    T.Resize(256), T.CenterCrop(224),          # taille fixe
    T.ToTensor(),                               # → (3, 224, 224), valeurs dans [0, 1]
    T.Normalize(mean=[0.485, 0.456, 0.406],     # standardisation par canal
                std=[0.229, 0.224, 0.225]),
])
x = transform(img).unsqueeze(0)                 # ajout de la dimension batch → (1, 3, 224, 224)
```

9.2 Convolution, filtres et partage des poids

La **convolution** consiste à faire glisser un petit tableau de poids, le **filtre** (ou *noyau*, *kernel*), sur toute l'image. À chaque position, on calcule le produit élément par élément entre le filtre et la zone couverte, puis on somme : le résultat est un nombre qui mesure à quel point la zone « ressemble » au motif du filtre.

FORMULE — CONVOLUTION 2D (COUCHE CONV2D)

Pour une entrée $X \in \mathbb{R}^{C_{in} \times H \times W}$, un filtre $W^{(k)} \in \mathbb{R}^{C_{in} \times K \times K}$ et un biais b_k, la sortie (carte de caractéristiques k) vaut :

$$Y_k[i,j] \;=\; b_k + \sum_{c=1}^{C_{in}} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} W_c^{(k)}[u,v]\; X_c[\, i \cdot S + u,\; j \cdot S + v \,]$$

où S est le pas (*stride*). En toute rigueur, il s'agit d'une *corrélation croisée* (le filtre n'est pas retourné) ; comme les poids sont appris, la distinction n'a aucune importance pratique.

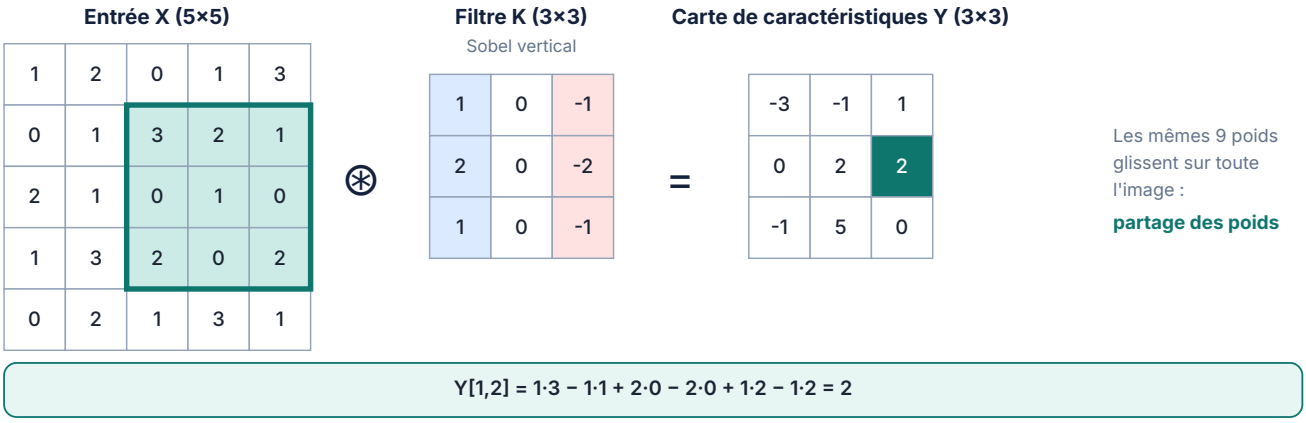


Figure 9.2 — Convolution d'une entrée 5×5 par un filtre de Sobel 3×3 (stride 1, sans padding). La fenêtre encadrée produit la case Y[1,2].

Les filtres sont des détecteurs de motifs

Le filtre de Sobel de la figure répond fortement à une **transition verticale** d'intensité (bord vertical). En traitement d'image classique, ces filtres sont conçus à la main (Sobel, Prewitt, flou gaussien, Laplacien). Dans un CNN, **les valeurs des filtres sont des paramètres appris** par rétropropagation : le réseau découvre lui-même les motifs utiles à la tâche. Une couche possède généralement des dizaines à des centaines de filtres, chacun produisant sa propre carte de caractéristiques (*feature map*).

Deux principes fondamentaux

CONNECTIVITÉ LOCALE

Chaque neurone de sortie ne regarde qu'une petite zone $K \times K$ de l'entrée : son **champ réceptif**. Les pixels proches étant fortement corrélés, l'information pertinente est d'abord locale (bords, coins, textures).

PARTAGE DES POIDS

Le **même filtre** est appliqué à toutes les positions. Un détecteur de bord utile en haut à gauche l'est aussi en bas à droite. Le nombre de paramètres ne dépend donc **pas** de la taille de l'image.

Conséquence mathématique : la convolution est **équivariante par translation**. Si l'on décale l'entrée, la carte de sortie est décalée de la même façon : $\mathrm{Conv}(\tau_\delta X) = \tau_\delta \,\mathrm{Conv}(X)$. Associée au pooling, elle confère une **invariance approximative** aux petites translations : un chat reste un chat, où qu'il soit dans l'image.

NOMBRE DE PARAMÈTRES D'UNE COUCHE CONVOLUTIVE

$$\#\text{params} = \underbrace{C_{out} \times C_{in} \times K \times K}_{\text{poids}} + \underbrace{C_{out}}_{\text{biais}}$$

Exemple : `Conv2d(3, 64, kernel_size=3)` $\rightarrow$ $64 \times 3 \times 3 \times 3 + 64 = 1\,792$ paramètres, quelle que soit la résolution de l'image. Comparez aux ≈ 150 millions de la couche dense évoquée en 9.1.

9.3 Canaux, stride, padding et pooling

Canaux d'entrée et de sortie

Un filtre traverse **toute la profondeur** de l'entrée : pour une entrée à C_{in} canaux, chaque filtre a la forme $C_{in} \times K \times K$ et combine l'information de tous les canaux. Une couche avec C_{out} filtres produit C_{out} cartes, qui deviennent les canaux de la couche suivante. Le tenseur de poids d'une couche a donc la forme `(C_out, C_in, K, K)`. Au fil du réseau, les canaux ne représentent plus des couleurs mais des **caractéristiques abstraites** (« présence d'un œil », « texture de fourrure »…).

Stride (pas) et padding (remplissage)

- **Stride** S : déplacement du filtre entre deux positions. $S = 2$ divise (environ) la résolution par 2 : c'est un sous-échantillonnage appris.
- **Padding** P : ajout de P rangées de zéros autour de l'entrée. Il évite que l'image rétrécisse à chaque couche et permet aux pixels du bord d'être traités autant que ceux du centre. Le padding « *same* » ($P = \lfloor K/2 \rfloor$ avec $S = 1$) conserve la taille ; le padding « *valid* » ($P = 0$) n'en ajoute pas.

FORMULE — TAILLE DE LA SORTIE

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1 \qquad (\text{idem pour } W_{out})$$

Exemple (couche d'entrée de ResNet) : $H_{in} = 224$, $K = 7$, $S = 2$, $P = 3$ → $\lfloor (224 + 6 - 7)/2 \rfloor + 1 = 112$.

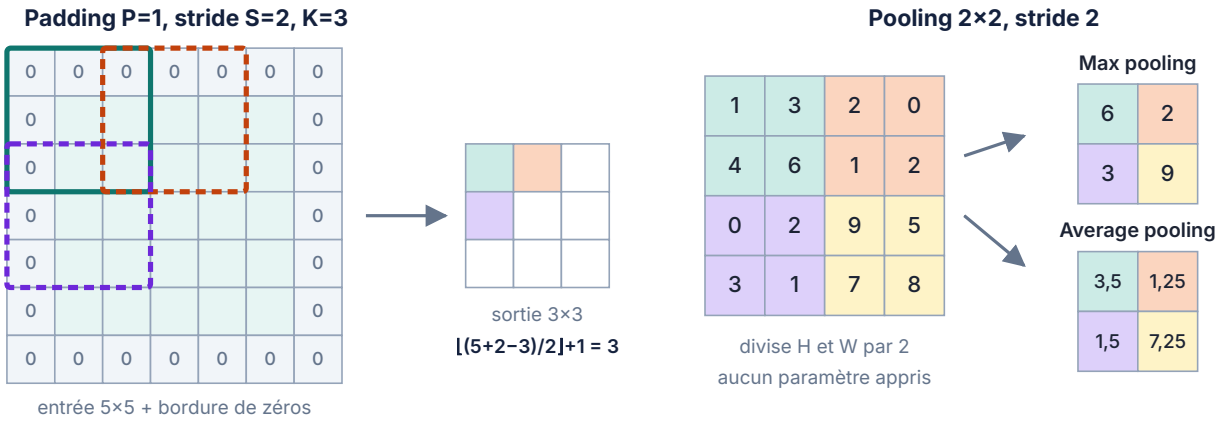


Figure 9.3 — À gauche : padding de 1 et stride de 2 (trois positions successives du filtre). À droite : max pooling et average pooling 2×2.

Pooling (sous-échantillonnage)

Le pooling résume chaque fenêtre (souvent 2×2, stride 2) par une statistique, **canal par canal**, sans aucun paramètre appris :

- **Max pooling** : $Y[i,j] = \max_{(u,v) \in \text{fenêtre}} X[2i+u,\, 2j+v]$. Conserve la réponse la plus forte (« le motif est-il présent dans la zone ? »).
- **Average pooling** : moyenne de la fenêtre ; plus lisse.
- **Global Average Pooling (GAP)** : moyenne de chaque carte entière, $\mathbb{R}^{C \times H \times W} \to \mathbb{R}^{C}$. Remplace l'aplatissement + grandes couches denses dans les architectures modernes (moins de paramètres, indépendant de la résolution d'entrée).

Le pooling réduit le coût de calcul (division par 4 du nombre de positions), augmente le champ réceptif des couches suivantes et apporte une robustesse aux petites déformations.

Champ réceptif et convolution 1×1

En empilant les couches, chaque neurone « voit » une zone de plus en plus grande de l'image d'origine. Avec $j_l = \prod_{i \le l} S_i$ le pas cumulé :

$$r_l = r_{l-1} + (K_l - 1)\, j_{l-1}, \qquad r_0 = 1,\ j_0 = 1.$$

Deux convolutions 3×3 empilées ont un champ réceptif 5×5 et trois en ont un de 7×7, avec moins de paramètres ($3 \times 9C^2 = 27C^2$ contre $49C^2$) et davantage de non-linéarités : c'est l'idée clé de VGG. La **convolution** 1×1 ne regarde aucun voisin : elle mélange les canaux en chaque pixel (combinaison linéaire), ce qui sert à réduire ou augmenter la profondeur à faible coût (blocs « bottleneck » de ResNet, Inception).

9.4 Construction et entraînement d'un CNN

Architecture type

Un CNN de classification enchaîne deux parties :

1. un **extracteur de caractéristiques** (*backbone*) : répétition de blocs `[Conv → BatchNorm → ReLU] (×1 à 3) → Pooling`. À chaque étage, la résolution spatiale diminue et le nombre de canaux augmente (souvent doublé) ;
2. une **tête de classification** : Global Average Pooling (ou aplatissement) → une ou plusieurs couches linéaires → logits → softmax.

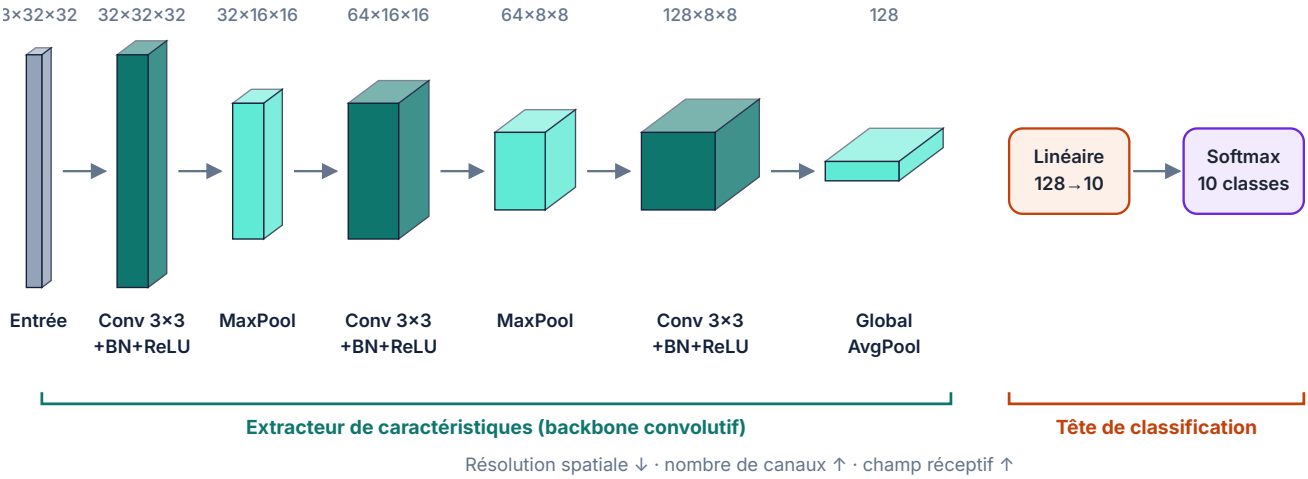


Figure 9.4 — CNN pour CIFAR-10 (images 32×32, 10 classes). Les dimensions sont indiquées au format C×H×W.

Cette organisation crée une **hiérarchie de représentations** : les premières couches détectent des bords et des couleurs, les couches intermédiaires des textures et des formes simples, les couches profondes des parties d'objets puis des objets entiers (visualisations de Zeiler & Fergus, 2014).

Implémentation PyTorch

```python
import torch
import torch.nn as nn

def conv_block(c_in, c_out):
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, kernel_size=3, padding=1, bias=False),  # biais inutile avant BN
        nn.BatchNorm2d(c_out),
        nn.ReLU(inplace=True),
    )

class SmallCNN(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            conv_block(3, 32),   nn.MaxPool2d(2),     # 32x32 → 16x16
            conv_block(32, 64),  nn.MaxPool2d(2),     # 16x16 → 8x8
            conv_block(64, 128),
            nn.AdaptiveAvgPool2d(1),                  # Global Average Pooling → (N, 128, 1, 1)
        )
        self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, n_classes))

    def forward(self, x):                             # x : (N, 3, 32, 32)
        return self.classifier(self.features(x))      # logits : (N, 10)

model = SmallCNN()
print(sum(p.numel() for p in model.parameters()))     # 94 762 paramètres
```

Couche	Sortie (C×H×W)	Paramètres	Champ réceptif
Entrée	3×32×32	—	1
Conv 3×3 (3→32) + BN	32×32×32	864 + 64	3
MaxPool 2×2	32×16×16	0	4
Conv 3×3 (32→64) + BN	64×16×16	18 432 + 128	8
MaxPool 2×2	64×8×8	0	10
Conv 3×3 (64→128) + BN	128×8×8	73 728 + 256	18
Global Avg Pool + Linear	10	1 290	image entière

Entraînement

- **Perte** : entropie croisée sur les logits, $\mathcal{L} = -\frac{1}{N}\sum_{n} \log \mathrm{softmax}(z_n)_{y_n}$ (`nn.CrossEntropyLoss` applique le softmax en interne).
- **Optimiseur** : SGD avec momentum 0,9 et *weight decay* $5 \cdot 10^{-4}$ (référence en vision), ou Adam/AdamW (10^{-3}) pour converger vite. Planification du taux d'apprentissage : *cosine annealing* ou *OneCycle*.
- **Augmentation de données** (sur l'entraînement uniquement) : recadrage aléatoire, retournement horizontal, variations de couleur, *RandAugment*, *Mixup/CutMix*. C'est le régularisateur le plus efficace en vision.
- **BatchNorm** stabilise et accélère l'apprentissage ; penser à `model.train()` / `model.eval()`, car BN et Dropout se comportent différemment.

```python
train_tf = T.Compose([T.RandomCrop(32, padding=4), T.RandomHorizontalFlip(), T.ToTensor(),
                      T.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616))])
device = "cuda" if torch.cuda.is_available() else "cpu"
model = SmallCNN().to(device)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9, weight_decay=5e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)

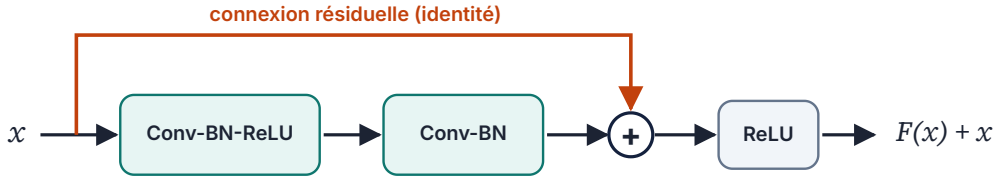
for epoch in range(30):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        loss = criterion(model(x), y)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
    scheduler.step()
    # ... boucle de validation avec model.eval() et torch.no_grad()
```

Architectures de référence

Modèle	Année	Idée clé	Paramètres
LeNet-5	1998	Premier CNN opérationnel (chiffres manuscrits) : conv → pool → dense.	≈ 60 k
AlexNet	2012	ReLU, Dropout, GPU ; remporte ImageNet avec un écart spectaculaire.	61 M
VGG-16	2014	Uniquement des convolutions 3×3 empilées ; profondeur 16.	138 M
Inception / GoogLeNet	2014	Branches parallèles 1×1, 3×3, 5×5 ; convolutions 1×1 de réduction.	6,8 M
ResNet-50	2015	Connexions résiduelles : réseaux de plus de 100 couches entraînables.	25,6 M
MobileNet / EfficientNet	2017–19	Convolutions séparables en profondeur ; mise à l'échelle équilibrée.	4–66 M
ConvNeXt	2022	CNN « modernisé » avec les recettes des Transformers.	28–198 M

LE BLOC RÉSIDUEL (RESNET)

Au lieu d'apprendre directement une transformation $H(x)$, le bloc apprend un **résidu** $F(x) = H(x) - x$ et calcule $y = \mathrm{ReLU}\left(F(x) + x\right)$. Le gradient traverse le raccourci sans atténuation, $\frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + I$, ce qui résout le problème de dégradation des réseaux très profonds.



Le bloc apprend un résidu F(x) ; le gradient circule directement par le raccourci.

Figure 9.5 — Bloc résiduel : la sortie est la somme de la transformation apprise et de l'entrée.

9.5 Comparaison entre MLP et CNN

Un MLP et un CNN sont tous deux des approximateurs universels ; la différence tient à leur **biais inductif**, c'est-à-dire aux hypothèses intégrées dans l'architecture. Le CNN suppose que l'information est *locale* et que les motifs peuvent apparaître *n'importe où* : hypothèses vraies pour les images, qui le rendent beaucoup plus efficace.

Critère	MLP (perceptron multicouche)	CNN (réseau convolutif)
Entrée	Vecteur aplati : la structure 2D est perdue.	Tenseur C×H×W : la topologie est conservée.
Connexions	Totales : chaque neurone voit tous les pixels.	Locales : champ réceptif K×K.
Poids	Un poids distinct par connexion.	Partagés sur toutes les positions.
Paramètres (CIFAR-10)	$3072 \to 512 \to 256 \to 10$: **1,71 M**	SmallCNN (9.4) : **0,095 M**
Translation	Aucune invariance : un objet décalé est une entrée « nouvelle ».	Équivariance (conv) + invariance approximative (pooling).
Taille d'entrée	Fixe.	Variable (avec GAP).
Précision CIFAR-10 typique	≈ 50–60 %	≈ 85–90 % (petit CNN), > 95 % (ResNet)
Usage adapté	Données tabulaires, vecteurs de caractéristiques, têtes de classification.	Images, vidéo, spectrogrammes audio, signaux 1D, grilles.

À NOTER

Les **Vision Transformers** (ViT, partie 11) ont un biais inductif plus faible encore que les CNN : ils découpent l'image en patchs de 16×16 traités comme des tokens. Ils dépassent les CNN avec *beaucoup* de données de préentraînement, mais les CNN restent plus efficaces sur des petits jeux de données et sur matériel embarqué.

9.6 Utilisation d'un modèle préentraîné

Entraîner un CNN performant depuis zéro exige des millions d'images étiquetées et des heures ou jours de GPU. Un **modèle préentraîné** a déjà appris ces représentations sur un grand jeu de données, typiquement **ImageNet-1k** (1,28 million d'images, 1 000 classes). On peut l'utiliser directement pour prédire, ou comme point de départ d'une nouvelle tâche (apprentissage par transfert).

APPRENTISSAGE PAR TRANSFERT

Réutiliser les paramètres θ^*_{source} appris sur une tâche source (grande, générique) comme **initialisation** pour une tâche cible (souvent petite). Les caractéristiques génériques (bords, textures, formes) se transfèrent ; seules les couches spécifiques doivent être adaptées.

Sources de modèles

- `torchvision.models` : ResNet, EfficientNet, ConvNeXt, ViT, MobileNet… avec des poids versionnés.
- `timm` (PyTorch Image Models) : plus de 1 000 architectures et poids.
- **Hugging Face Hub** : modèles de vision (ViT, DINOv2, CLIP, ConvNeXt).

Règle d'or : reproduire le prétraitement d'origine

Le modèle n'a « vu » que des images redimensionnées et normalisées d'une certaine manière (taille, moyenne, écart-type, ordre RGB). Il faut appliquer **exactement le même prétraitement** en inférence, sinon les performances chutent fortement. Avec torchvision, l'objet `weights.transforms()` fournit ce prétraitement.

```python
import torch
from torchvision.models import resnet50, ResNet50_Weights

weights = ResNet50_Weights.IMAGENET1K_V2
model = resnet50(weights=weights).eval()          # mode évaluation : BN figée, pas de dropout
preprocess = weights.transforms()                 # prétraitement exact utilisé à l'entraînement

x = preprocess(img).unsqueeze(0)                  # (1, 3, 224, 224)
with torch.no_grad():                             # pas de graphe de calcul → plus rapide
    probs = model(x).softmax(dim=1)
top5 = probs.topk(5)
for p, i in zip(top5.values[0], top5.indices[0]):
    print(f"{weights.meta['categories'][i]:<20} {p:.1%}")
```

PIÈGES FRÉQUENTS

- Oublier `model.eval()` : la BatchNorm utilise alors les statistiques du lot courant, prédictions instables.
- Normalisation différente de celle du préentraînement, ou images en BGR (OpenCV).
- Vérifier la **licence** des poids et le domaine d'origine : un modèle ImageNet est peu adapté, tel quel, aux radiographies.

9.7 Extraction de représentations et fine-tuning

Un CNN préentraîné se décompose en un backbone ϕ_θ (tout sauf la dernière couche) et une tête g_ψ. Le backbone transforme une image en un **vecteur de représentation** (*embedding*) $h = \phi_\theta(x) \in \mathbb{R}^d$ ($d = 2048$ pour ResNet-50, 512 pour ResNet-18). On remplace la tête d'origine (1 000 classes ImageNet) par une nouvelle tête adaptée aux K classes cibles, puis on choisit ce qu'on entraîne.

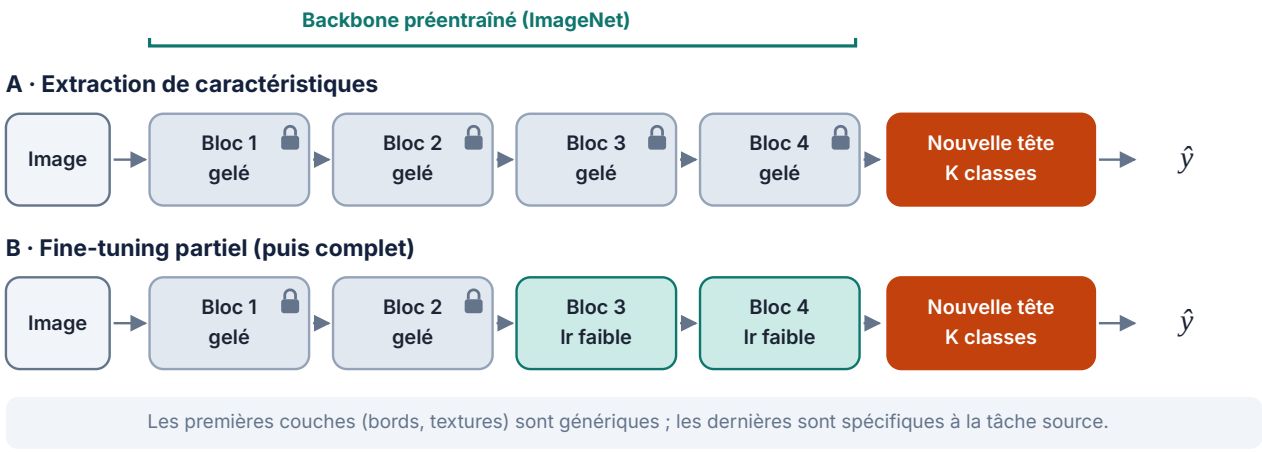


Figure 9.6 — Deux stratégies de transfert : backbone gelé (A) ou réentraîné partiellement avec un faible taux d'apprentissage (B).

Stratégie A — Extraction de caractéristiques (*feature extraction*)

Tous les paramètres du backbone sont **gelés** (θ fixe) ; seule la nouvelle tête est entraînée :

$$\min_{\psi} \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}\big(g_\psi(\phi_{\theta^*}(x_n)),\, y_n\big).$$

Rapide, peu gourmand en mémoire, très robuste au surapprentissage : idéal avec quelques centaines d'images. On peut même pré-calculer les embeddings une fois pour toutes et entraîner un classifieur simple (régression logistique, k-NN, SVM). Les embeddings servent aussi à la recherche d'images similaires (similarité cosinus) ou au clustering.

```python
import torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
for p in model.parameters():
    p.requires_grad = False                      # gel du backbone
model.fc = nn.Linear(model.fc.in_features, 5)    # nouvelle tête : 2048 → 5 classes (entraînable)

optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# Extraction d'embeddings seuls : model.fc = nn.Identity() → sortie (N, 2048)
```

Stratégie B — Fine-tuning (ajustement fin)

On **dégèle** tout ou partie du backbone et on poursuit l'entraînement sur la tâche cible, avec un taux d'apprentissage **10 à 100 fois plus faible** qu'à l'entraînement initial, pour ajuster les représentations sans détruire ce qui a été appris (*oubli catastrophique*).

1. Entraîner d'abord la tête seule quelques époques (backbone gelé) : évite que les gradients d'une tête aléatoire n'abîment le backbone.
2. Dégeler progressivement les blocs, du plus haut au plus bas (*gradual unfreezing*).
3. Utiliser des **taux d'apprentissage différenciés** : faible pour les couches basses (génériques), plus élevé pour les couches hautes et la tête.

```python
for p in model.parameters():
    p.requires_grad = True                          # dégel
optimizer = torch.optim.AdamW([
    {"params": model.layer3.parameters(), "lr": 1e-5},   # couches intermédiaires : très lent
    {"params": model.layer4.parameters(), "lr": 1e-4},   # couches hautes : lent
    {"params": model.fc.parameters(),     "lr": 1e-3},   # tête : normal
], weight_decay=1e-4)
# conv1, layer1 et layer2 restent gelés (non passés à l'optimiseur, requires_grad=False conseillé)
```

Quelle stratégie choisir ?

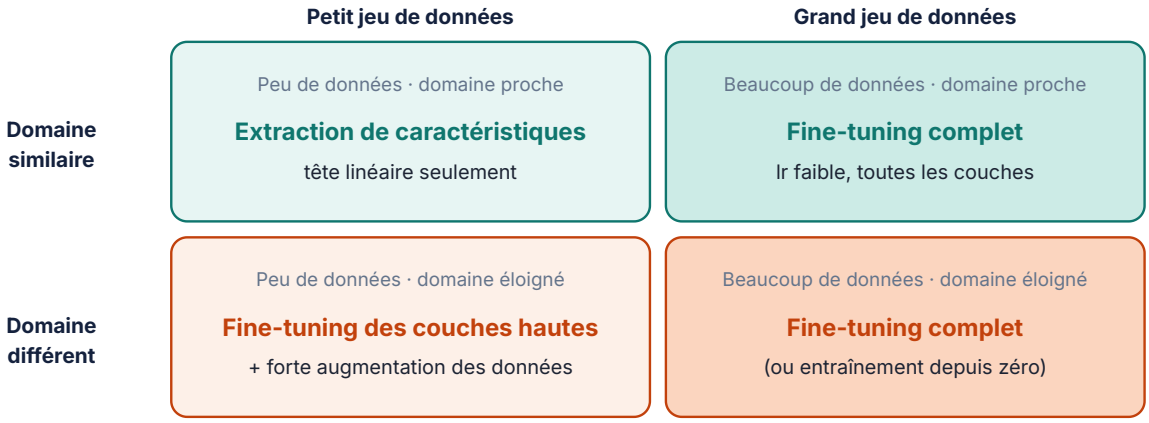


Figure 9.7 — Choix de la stratégie selon la taille du jeu cible et sa ressemblance avec le domaine source.

BONNES PRATIQUES

- Garder la même résolution et normalisation que le modèle source ; utiliser une augmentation de données adaptée au domaine.
- Sur de très petits lots, garder les couches BatchNorm en mode `eval()` (statistiques gelées) pendant le fine-tuning.
- Toujours comparer à une **référence** : tête linéaire sur backbone gelé. Si le fine-tuning ne fait pas mieux, il surapprend.
- Surveiller la validation et utiliser l'arrêt anticipé ; le fine-tuning converge en peu d'époques (5 à 20).

À RETENIR — PARTIE 9

- Une image est un tenseur $C \times H \times W$; on la redimensionne, on la met à l'échelle et on la normalise par canal.
- La convolution applique un même petit filtre appris sur toute l'image : connectivité locale, partage des poids, équivariance par translation.
- Taille de sortie : $\lfloor (H + 2P - K)/S \rfloor + 1$; le pooling sous-échantillonne sans paramètres ; le champ réceptif croît avec la profondeur.
- CNN = backbone [Conv-BN-ReLU-Pool] + tête (GAP + Linear) ; les connexions résiduelles permettent la grande profondeur.
- Un CNN est 10 à 100 fois plus économe en paramètres qu'un MLP et bien plus précis sur les images.
- Transfert : extraction de caractéristiques (backbone gelé) avec peu de données ; fine-tuning à faible taux d'apprentissage sinon.

PARTIE 10

Données séquentielles et réseaux récurrents

10

Texte, parole, capteurs, cours financiers : dans une séquence, l'ordre porte l'information. Les réseaux récurrents (RNN) traitent les éléments un à un en maintenant une mémoire, l'état caché. Les LSTM et GRU ajoutent des portes qui contrôlent cette mémoire pour capturer des dépendances longues.

10.1 Séquences et séries temporelles
10.2 Fenêtres temporelles et préparation des données
10.3 Réseaux récurrents (RNN)
10.4 État caché et dépendances temporelles
10.5 Introduction aux LSTM et GRU
10.6 Évaluation avec un découpage chronologique

10.1 Séquences et séries temporelles

DÉFINITIONS

Une **séquence** est une suite ordonnée $x_{1:T} = (x_1, x_2, \ldots, x_T)$ avec $x_t \in \mathbb{R}^F$ (F caractéristiques par pas de temps). L'ordre est essentiel : permuter les éléments change le sens. Une **série temporelle** est une séquence dont les observations sont indexées par le temps, généralement à intervalles réguliers (pas horaire, journalier…).

Contrairement aux données tabulaires, les observations ne sont **pas indépendantes** : x_t dépend de son passé. La longueur T peut aussi varier d'un exemple à l'autre (phrases de longueurs différentes).

Domaine	Exemple de séquence	Tâche typique
Langage	Suite de mots / tokens	Classification de sentiment, traduction, génération
Audio	Échantillons ou trames de spectrogramme	Reconnaissance vocale
Capteurs / IoT	Température, vibrations, consommation électrique	Prévision, détection d'anomalies
Finance	Prix, volumes	Prévision, gestion du risque
Santé	ECG, séjours hospitaliers	Diagnostic, prédiction d'événements

Types de tâches séquentielles

- **Plusieurs-vers-un** (*many-to-one*) : une séquence → une sortie (sentiment d'une critique, prévision de la valeur suivante).
- **Plusieurs-vers-plusieurs synchronisé** : une sortie par pas (étiquetage morpho-syntaxique, segmentation).
- **Séquence-vers-séquence** (*seq2seq*) : longueurs d'entrée et de sortie différentes (traduction, prévision multi-horizon).

- **Un-vers-plusieurs** : une entrée → une séquence (légende d'image, génération musicale).

Composantes d'une série temporelle

On décompose souvent une série en $y_t = \underbrace{T_t}_{\text{tendance}} + \underbrace{S_t}_{\text{saisonnalité}} + \underbrace{\varepsilon_t}_{\text{bruit}}$ (modèle additif ; multiplicatif si l'amplitude saisonnière croît avec le niveau).

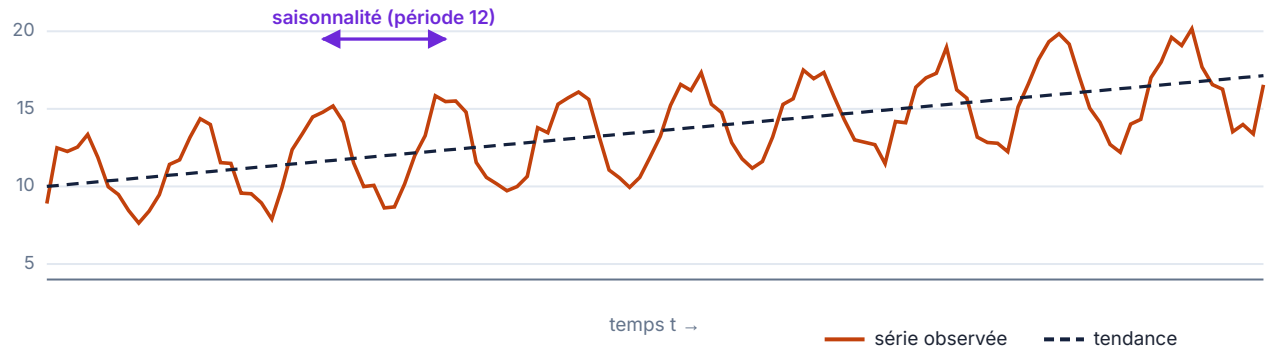


Figure 10.1 — Série mensuelle simulée : tendance linéaire + saisonnalité de période 12 + bruit.

- **Stationnarité** : moyenne, variance et autocorrélations constantes dans le temps. Beaucoup de méthodes la supposent ; on s'en approche par différenciation ($\Delta y_t = y_t - y_{t-1}$) ou par passage au logarithme.
- **Autocorrélation** au décalage k : mesure la dépendance entre la série et sa version retardée. Un pic à $k = 12$ révèle une saisonnalité annuelle sur des données mensuelles.

$$\rho(k) = \frac{\sum_{t=1}^{T-k} (y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_{t=1}^{T} (y_t - \bar{y})^2}$$

- Séries **univariées** ($F = 1$) ou **multivariées** (plusieurs variables, plus des covariables connues à l'avance : calendrier, jours fériés, météo prévue).

10.2 Fenêtres temporelles et préparation des données

Un réseau de neurones apprend sur des couples (X, y). Pour une série unique, on crée ces couples par **fenêtres glissantes** : on prend L pas passés en entrée (*lookback*) et les H pas suivants comme cible (*horizon*).

CONSTRUCTION DES EXEMPLES

$$X^{(n)} = (x_n, \dots, x_{n+L-1}) \in \mathbb{R}^{L \times F}, \qquad y^{(n)} = (y_{n+L}, \dots, y_{n+L+H-1}) \in \mathbb{R}^{H}$$

Avec un glissement de 1, une série de longueur T donne $N = T - L - H + 1$ exemples, empilés en un tenseur (N, L, F) : format `batch_first` attendu par `nn.RNN`, `nn.LSTM`, `nn.GRU`.

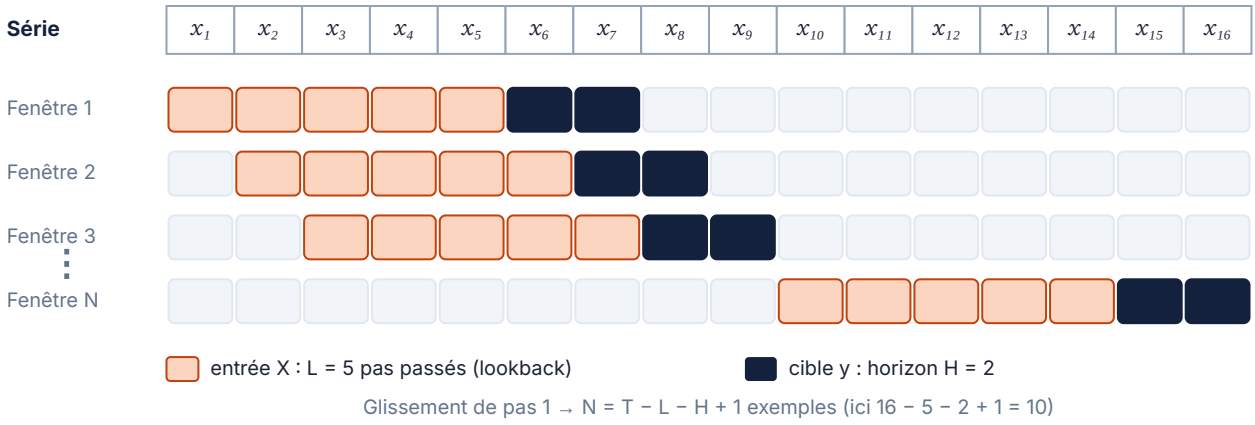


Figure 10.2 — Fenêtres glissantes : L = 5 pas d'historique servent à prédire les H = 2 pas suivants.

```python
import numpy as np

def make_windows(series, L, H, target_col=0):
    """series : (T, F) → X : (N, L, F), y : (N, H)"""
    T = len(series)
    X = np.stack([series[i:i + L] for i in range(T - L - H + 1)])
    y = np.stack([series[i + L:i + L + H, target_col] for i in range(T - L - H + 1)])
    return X.astype(np.float32), y.astype(np.float32)

# 1) découpage chronologique AVANT tout calcul de statistique
n = len(data); i_tr, i_va = int(0.7 * n), int(0.85 * n)
train, val, test = data[:i_tr], data[i_tr - L:i_va], data[i_va - L:]   # recouvrement de L pas pour l'historique
# 2) normalisation ajustée sur l'entraînement uniquement
mu, sd = train.mean(axis=0), train.std(axis=0)
train, val, test = [(d - mu) / sd for d in (train, val, test)]
# 3) fenêtres
X_tr, y_tr = make_windows(train, L=48, H=24)
```

Choix et étapes clés

- **Longueur** L : doit couvrir les dépendances utiles (au moins une période saisonnière). Trop courte : information manquante ; trop longue : bruit, coût, gradients difficiles pour un RNN.
- **Normalisation** : standardisation ou min-max, **ajustée sur l'entraînement seulement** (sinon fuite d'information du futur). On inverse la transformation pour exprimer les erreurs dans l'unité d'origine.
- **Caractéristiques temporelles** : encodage cyclique de l'heure ou du mois, $\left(\sin\frac{2\pi h}{24}, \cos\frac{2\pi h}{24}\right)$, qui place 23 h à côté de 0 h ; retards (*lags*) ; moyennes mobiles calculées uniquement avec le passé.
- **Valeurs manquantes** : interpolation, report de la dernière valeur (*forward fill*), éventuellement un indicateur binaire de valeur manquante.
- **Prévision multi-pas** : *stratégie directe* (le modèle sort les H valeurs d'un coup) ou *récursive* (on prédit $t+1$, on réinjecte la prédiction, etc. ; les erreurs s'accumulent).

Séquences de longueur variable (texte)

Pour regrouper des phrases de longueurs différentes dans un lot, on les **complète** (*padding*) avec un token spécial jusqu'à la longueur maximale du lot, et on indique au réseau d'ignorer ces

positions : masque, ou `nn.utils.rnn.pack_padded_sequence` qui évite au RNN de traiter le remplissage.

10.3 Réseaux récurrents (RNN)

Un MLP traite une entrée de taille fixe et n'a aucune mémoire. Un **réseau récurrent** lit la séquence pas à pas et met à jour un vecteur interne, l'**état caché** $h_t \in \mathbb{R}^{d_h}$, qui résume tout ce qui a été vu jusque-là. La même cellule, avec les **mêmes poids**, est appliquée à chaque instant : c'est le partage des poids *dans le temps* (comme la convolution partage ses poids dans l'espace).

ÉQUATIONS DU RNN SIMPLE (ELMAN, 1990)

$$\boxed{h_t = \tanh\big(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\big)} \qquad \hat{y}_t = W_{hy}\, h_t + b_y$$

avec $W_{xh} \in \mathbb{R}^{d_h \times F}$, $W_{hh} \in \mathbb{R}^{d_h \times d_h}$, $W_{hy} \in \mathbb{R}^{d_{out} \times d_h}$ et $h_0 = 0$. Nombre de paramètres de la cellule : $d_h(F + d_h) + 2d_h$ dans PyTorch (deux vecteurs de biais), **indépendant de** T.

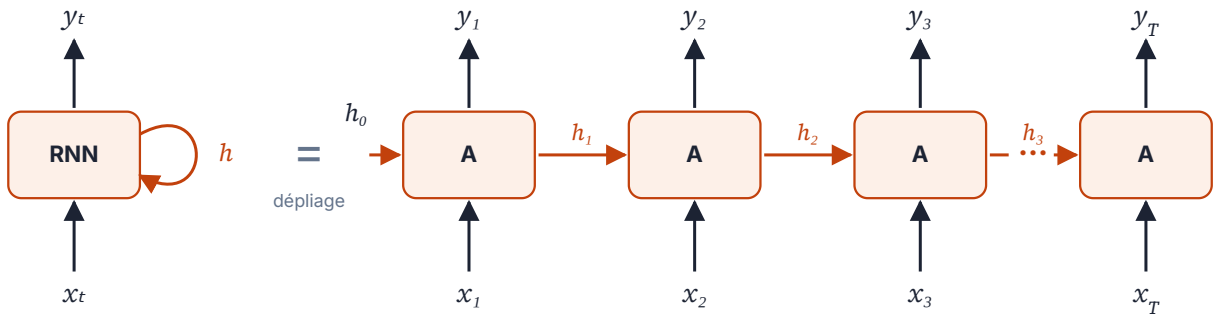


Figure 10.3 — Un RNN replié (boucle de récurrence) est équivalent à un réseau profond « déplié » dans le temps dont toutes les couches partagent les poids.

Pour une tâche *many-to-one* (classification, prévision), on n'utilise que le dernier état h_T (ou un pooling sur tous les h_t). Pour une tâche synchronisée, on utilise chaque $\hat{y}_t$.

```python
import torch.nn as nn

rnn = nn.RNN(input_size=8, hidden_size=64, num_layers=1, batch_first=True)
x = torch.randn(32, 48, 8)        # (N, L, F) : 32 fenêtres de 48 pas, 8 variables
out, h_n = rnn(x)
print(out.shape)                  # (32, 48, 64) : h_t pour chaque pas t
print(h_n.shape)                  # (1, 32, 64)  : dernier état h_T (par couche)
```

Variantes

- **RNN empilé** (`num_layers=2` ou plus) : la séquence des h_t d'une couche est l'entrée de la suivante ; représentations plus abstraites.
- **RNN bidirectionnel** (`bidirectional=True`) : un RNN lit de gauche à droite, un autre de droite à gauche ; on concatène $[\overrightarrow{h_t}; \overleftarrow{h_t}]$. Utile quand toute la séquence est disponible (texte), **interdit** en prévision (cela reviendrait à regarder le futur).

- **Encodeur-décodeur** (seq2seq) : un RNN encode la source en un vecteur, un second RNN génère la cible ; ancêtre direct des Transformers (partie 11).

10.4 État caché et dépendances temporelles

L'état caché est une **mémoire compressée de taille fixe** : $h_t = f(h_{t-1}, x_t)$ dépend récursivement de tout le passé $x_{1:t}$. Il doit encoder ce qui sera utile plus tard (le sujet d'une phrase pour accorder un verbe vingt mots après, la tendance récente d'une série…). Une **dépendance à long terme** relie une sortie à une entrée lointaine ; c'est là que le RNN simple échoue.

Rétropropagation dans le temps (BPTT)

On entraîne le RNN déplié comme un réseau profond : la perte totale $\mathcal{L} = \sum_t \mathcal{L}_t$ est rétropropagée à travers tous les pas de temps, et les gradients des poids partagés s'additionnent sur les instants. Le signal reliant l'instant t à un instant antérieur k fait intervenir un **produit de jacobiennes** :

GRADIENTS ÉVANESCENTS ET EXPLOSIFS

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} \operatorname{diag}\left(1 - h_i^2\right) W_{hh}$$

En simplifiant, la norme de ce produit se comporte comme λ^{t-k}, où λ dépend de la plus grande valeur singulière de W_{hh} et de la dérivée de tanh (≤ 1) :

- $\lambda < 1$: le gradient **s'évanouit** exponentiellement, le réseau n'apprend pas les dépendances longues (en pratique au-delà de 10 à 20 pas) ;
- $\lambda > 1$: le gradient **explose**, entraînement instable (pertes NaN).

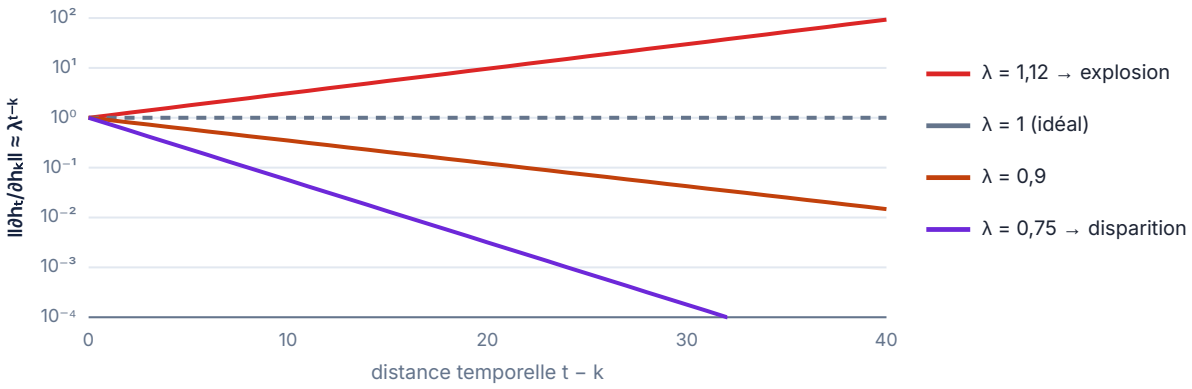


Figure 10.4 — Évolution d'un facteur λ^{t-k} (échelle logarithmique) : quelques dizaines de pas suffisent à annuler ou faire exploser le gradient.

Remèdes

- **Écrêtage du gradient** (*gradient clipping*) contre l'explosion : si $\|g\| > \tau$, $g \leftarrow \tau\, g / \|g\|$. En PyTorch : `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` entre `backward()` et `step()`.
- **BPTT tronquée** : on rétropropage sur des segments de longueur limitée (coût mémoire maîtrisé).

- **Initialisation** orthogonale ou identité de W_{hh}, normalisation.
- Contre l'évanouissement, la vraie solution est architecturale : des cellules à **portes** (LSTM, GRU) qui créent un chemin où l'information et le gradient circulent presque sans atténuation.

10.5 Introduction aux LSTM et GRU

LSTM — Long Short-Term Memory (Hochreiter & Schmidhuber, 1997)

Le LSTM ajoute à l'état caché h_t une **cellule mémoire** c_t, une « autoroute » de l'information mise à jour de façon *additive*. Trois **portes** à valeurs dans $[0, 1]$ (sigmoïdes) décident, composante par composante, quoi oublier, quoi écrire et quoi exposer.

ÉQUATIONS DU LSTM

$$
\begin{aligned}
f_t &= \sigma\big(W_f\,[h_{t-1}, x_t] + b_f\big) && \text{porte d'oubli}\\
i_t &= \sigma\big(W_i\,[h_{t-1}, x_t] + b_i\big) && \text{porte d'entrée}\\
\tilde{c}_t &= \tanh\big(W_c\,[h_{t-1}, x_t] + b_c\big) && \text{contenu candidat}\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{mise à jour de la mémoire}\\
o_t &= \sigma\big(W_o\,[h_{t-1}, x_t] + b_o\big) && \text{porte de sortie}\\
h_t &= o_t \odot \tanh(c_t) && \text{nouvel état caché}
\end{aligned}
$$

$[h_{t-1}, x_t]$: concaténation ; $\odot$: produit élément par élément ; $\sigma(z) = 1/(1+e^{-z})$.

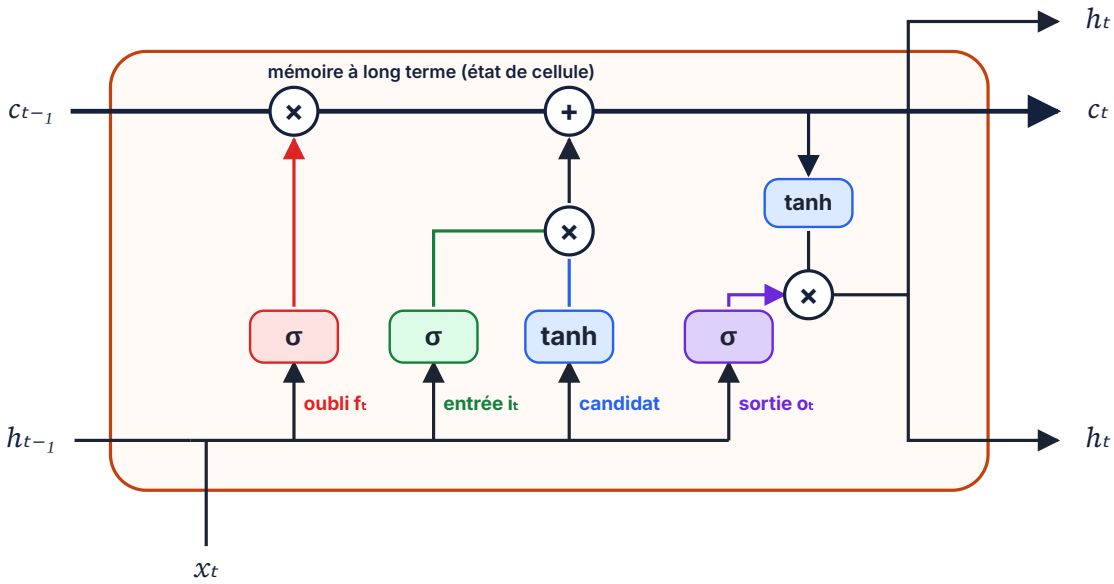


Figure 10.5 — Cellule LSTM. La ligne supérieure (état de cellule c) ne subit qu'une multiplication par la porte d'oubli et une addition.

Pourquoi ça marche : $\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t)$ (en ignorant les dépendances indirectes). Si la porte d'oubli reste proche de 1, le gradient traverse de nombreux pas presque intact : c'est le *carrousel d'erreur constante*. Le réseau apprend lui-même quand retenir et quand effacer. Astuce courante : initialiser b_f à 1 pour favoriser la mémorisation au début de l'entraînement.

GRU — Gated Recurrent Unit (Cho et al., 2014)

Version simplifiée : pas de cellule séparée, deux portes seulement.

ÉQUATIONS DU GRU

$$
\begin{aligned}
z_t &= \sigma\big(W_z\,[h_{t-1}, x_t] + b_z\big) && \text{porte de mise à jour} \\
r_t &= \sigma\big(W_r\,[h_{t-1}, x_t] + b_r\big) && \text{porte de réinitialisation} \\
\tilde{h}_t &= \tanh\big(W_h\,[r_t \odot h_{t-1},\, x_t] + b_h\big) && \text{état candidat} \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{interpolation}
\end{aligned}
$$

La porte z_t joue à la fois le rôle des portes d'oubli et d'entrée du LSTM. (PyTorch utilise la convention inverse : $h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$; c'est équivalent.)

Critère	RNN simple	LSTM	GRU
États	h_t	h_t et c_t	h_t
Portes	0	3 (oubli, entrée, sortie)	2 (mise à jour, réinitialisation)
Paramètres	$1 \times \left[d_h(F + d_h) + 2d_h\right]$	$4\times$	$3\times$
Dépendances longues	Faibles (≈ 10 pas)	Bonnes (centaines de pas)	Bonnes, proches du LSTM
Quand l'utiliser	Pédagogie, séquences très courtes	Séquences longues, beaucoup de données	Plus rapide, petits jeux de données

PYTHON

```python
class LSTMForecaster(nn.Module):
    def __init__(self, n_features, hidden=64, horizon=24, layers=2, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(n_features, hidden, num_layers=layers,
                            batch_first=True, dropout=dropout)
        self.head = nn.Linear(hidden, horizon)          # stratégie directe : H sorties

    def forward(self, x):                               # x : (N, L, F)
        out, (h_n, c_n) = self.lstm(x)                  # h_n : (layers, N, hidden)
        return self.head(h_n[-1])                       # dernier état de la dernière couche → (N, H)

model = LSTMForecaster(n_features=8)
loss = nn.MSELoss()(model(xb), yb)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)   # sécurité anti-explosion
optimizer.step()
```

POSITIONNEMENT ACTUEL

Pour le texte, les LSTM/GRU ont été largement remplacés par les Transformers (partie 11), parallélisables et meilleurs sur les dépendances longues. Ils restent compétitifs pour des séries temporelles de taille modeste, le traitement en flux (*streaming*, coût constant par pas) et l'embarqué. Des architectures récurrentes modernes (state space models comme Mamba, xLSTM) reprennent ces idées à grande échelle.

10.6 Évaluation avec un découpage chronologique

Le découpage aléatoire train/validation/test, standard pour des données i.i.d., est **incorrect** pour une série temporelle : le modèle s'entraînerait sur des observations postérieures à celles qu'il doit prédire. C'est une **fuite d'information** (*look-ahead bias*) qui donne des scores optimistes, impossibles à reproduire en production où l'on ne connaît que le passé.

PRINCIPE

Toujours **entraîner sur le passé, valider sur un futur proche, tester sur un futur plus lointain** : $t_{train} < t_{val} < t_{test}$. Toutes les étapes ajustées sur les données (normalisation, imputation, sélection de variables) le sont sur l'ensemble d'entraînement seul.

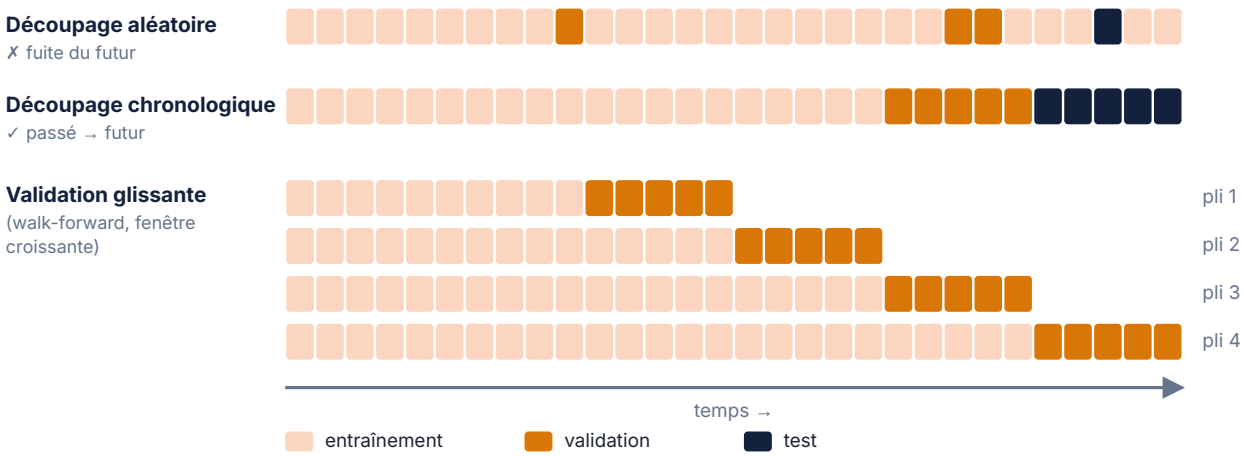


Figure 10.6 — Découpage aléatoire (à proscrire), découpage chronologique, et validation glissante à fenêtre croissante.

Validation glissante (*walk-forward / backtesting*)

Une seule coupure peut tomber sur une période atypique. On répète donc l'évaluation sur plusieurs origines successives : fenêtre d'entraînement **croissante** (*expanding*) ou **glissante** de taille fixe (*rolling*, adaptée si la dynamique évolue), puis on moyenne les scores. On peut laisser un **écart** (*gap*) entre entraînement et validation quand les fenêtres d'entrée se chevauchent ou que la cible est connue avec retard.

```python
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=4, test_size=500, gap=24)   # fenêtre croissante + écart
for k, (idx_tr, idx_va) in enumerate(tscv.split(X)):
    model = build_model()
    fit(model, X[idx_tr], y[idx_tr])                        # les indices de validation sont
toujours postérieurs
    scores.append(evaluate(model, X[idx_va], y[idx_va]))
print(np.mean(scores), np.std(scores))
```

Références naïves et métriques

Un modèle profond n'a de valeur que s'il bat des **références simples** :

- **Persistance** : $\hat{y}_{t+1} = y_t$; **naïf saisonnier** : $\hat{y}_{t+h} = y_{t+h-m}$ (m = période) ; moyenne mobile ; modèles statistiques (ARIMA, lissage exponentiel).

MÉTRIQUES DE PRÉVISION

$$\text{MAE} = \frac{1}{n}\sum_t |y_t - \hat{y}_t| \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_t (y_t - \hat{y}_t)^2} \qquad \text{MAPE} = \frac{100\%}{n}\sum_t \left|\frac{y_t - \hat{y}_t}{y_t}\right|$$

$$\text{MASE} = \frac{\text{MAE}_{\text{modèle}}}{\frac{1}{T-m}\sum_{t=m+1}^{T} |y_t - y_{t-m}|} \qquad (\text{MASE} < 1 \Leftrightarrow \text{meilleur que le naïf saisonnier})$$

MAE : robuste, dans l'unité de la cible. RMSE : pénalise davantage les grosses erreurs. MAPE : en pourcentage, mais indéfini si $y_t = 0$. MASE : comparable entre séries.

ERREURS CLASSIQUES

- Mélanger les fenêtres avant le découpage (`shuffle=True` sur l'ensemble complet). Le mélange est permis *à l'intérieur* de l'ensemble d'entraînement, une fois le découpage fait.
- Normaliser avec la moyenne de toute la série, ou calculer des moyennes mobiles centrées (qui utilisent le futur).
- Utiliser comme variable d'entrée une information qui ne sera pas disponible au moment de la prévision.
- Évaluer une prévision récursive avec les vraies valeurs réinjectées (*teacher forcing*) au lieu des prédictions.

À RETENIR — PARTIE 10

- Dans une séquence, l'ordre porte l'information et les observations sont dépendantes (tendance, saisonnalité, autocorrélation).
- Les fenêtres glissantes transforment une série en exemples $(N, L, F) \to (N, H)$; la normalisation est ajustée sur l'entraînement seul.
- RNN : $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$, poids partagés dans le temps, entraînement par BPTT.
- Le produit de jacobiennes provoque l'évanouissement ou l'explosion des gradients ; l'écrêtage traite l'explosion.
- LSTM (3 portes + cellule additive) et GRU (2 portes) capturent les dépendances longues.
- Évaluation : découpage chronologique, validation glissante, comparaison aux références naïves, MAE/RMSE/MASE.

PARTIE 11

Attention et Transformers

11

Le Transformer (Vaswani et al., 2017, « Attention Is All You Need ») abandonne la récurrence : chaque élément d'une séquence regarde directement tous les autres grâce au mécanisme d'attention. Entièrement parallélisable, il est à la base de BERT, GPT, T5, des grands modèles de langage et des Vision Transformers.

11.1 Tokenisation et embeddings
11.2 Mécanisme d'attention et auto-attention
11.3 Query, key et value
11.4 Attention à plusieurs têtes
11.5 Information de position
11.6 Architecture générale des Transformers
11.7 Encodeurs et décodeurs
11.8 Modèles préentraînés avec Hugging Face

11.1 Tokenisation et embeddings

Un réseau ne manipule que des nombres. Le traitement d'un texte commence donc par deux étapes : la **tokenisation** (texte → suite d'unités discrètes appelées *tokens* → identifiants entiers) puis l'**embedding** (identifiant → vecteur dense appris).

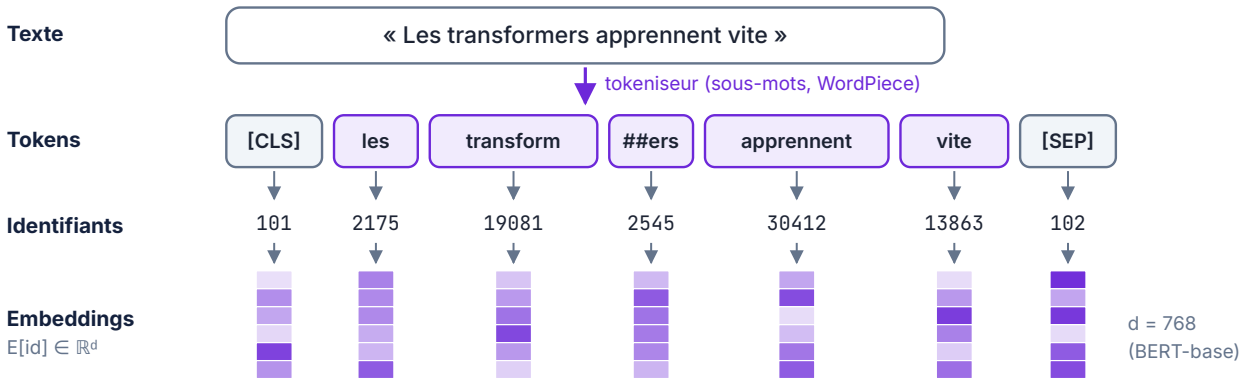


Figure 11.1 — Du texte aux vecteurs : découpage en sous-mots, conversion en identifiants (valeurs illustratives), puis lecture des embeddings.

Niveaux de tokenisation

Niveau	Exemple pour « apprennent »	Avantages	Inconvénients
Mot	[apprennent]	Tokens porteurs de sens	Vocabulaire énorme ; mots inconnus (OOV) remplacés par [UNK]
Caractère	[a, p, p, r, e, ...]	Vocabulaire minuscule, aucun mot inconnu	Séquences très longues, sens peu explicite
Sous-mot	[app, ##rennent]	Compromis : 30 k–250 k tokens, tout mot est représentable	Découpages parfois peu intuitifs

Les modèles modernes utilisent tous des **sous-mots** : les mots fréquents restent entiers, les mots rares sont décomposés en morceaux fréquents (racines, suffixes).

- **BPE** (*Byte-Pair Encoding* ; GPT, RoBERTa, Llama) : on part des caractères (ou des octets) et on **fusionne itérativement la paire adjacente la plus fréquente** du corpus, jusqu'à la taille de vocabulaire voulue. Ex. : `l o w`, `l o w e r`, `l o w e s t` → fusion `l+o` → `lo`, puis `lo+w` → `low`, puis `e+r`…
- **WordPiece** (BERT) : fusion de la paire qui maximise la vraisemblance du corpus ; les suites de mots sont préfixées par `##`.
- **Unigram / SentencePiece** (T5, CamemBERT, XLM-R) : travaille sur le texte brut (espaces encodés par `▁`), indépendant de la langue.

Le tokeniseur ajoute des **tokens spéciaux** : `[CLS]` (représentation de la phrase entière pour la classification), `[SEP]` (séparateur), `[PAD]` (remplissage), `[MASK]`, `<s>` / `</s>` (début et fin de séquence). Il fournit aussi un **masque d'attention** (1 = vrai token, 0 = remplissage). Un modèle doit **toujours** être utilisé avec son propre tokeniseur.

Embeddings

COUCHE D'EMBEDDING

Une matrice apprise $E \in \mathbb{R}^{|V| \times d}$ ($|V|$: taille du vocabulaire, d : dimension du modèle). Le token d'identifiant i est représenté par la ligne $e_i = E[i, :]$, ce qui équivaut à multiplier un vecteur *one-hot* par E : $e_i = \mathbf{1}_i^\top E$. Pour BERT-base : $30\,522 \times 768 \approx 23{,}4$ M paramètres.

Contrairement à l'encodage one-hot (vecteurs orthogonaux, aucune notion de proximité), les embeddings appris placent les tokens de sens voisin à proximité. On mesure cette proximité par la **similarité cosinus** $\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}$. Les embeddings statiques (Word2Vec, GloVe) donnent un seul vecteur par mot ; ceux d'un Transformer sont ensuite **contextualisés** par les couches d'attention (« avocat » le fruit ≠ « avocat » le juriste).

```python
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("bert-base-multilingual-cased")
enc = tok(["Les transformers apprennent vite"], padding=True, truncation=True,
          max_length=128, return_tensors="pt")
print(tok.convert_ids_to_tokens(enc["input_ids"][0]))   # ['[CLS]', 'Les', ..., '[SEP]'] (sous-mots préfixés par ##)
print(enc["attention_mask"])                            # 1 = token réel, 0 = padding
```

11.2 Mécanisme d'attention et auto-attention

Motivation : le goulot d'étranglement des RNN

Dans un encodeur-décodeur RNN (10.3), toute la phrase source est compressée dans **un seul vecteur** de taille fixe h_T : les phrases longues sont mal traduites. Bahdanau et al. (2015) proposent que le décodeur, à chaque étape, **regarde tous les états de l'encodeur** et choisisse les plus pertinents. C'est le mécanisme d'attention.

ATTENTION : FORME GÉNÉRALE

Étant donnés une requête q et des paires (clé k_j, valeur v_j) pour $j = 1, \dots, n$:

$$\underbrace{s_j = \text{score}(q, k_j)}_{\text{pertinence}} \qquad \underbrace{\alpha_j = \frac{e^{s_j}}{\sum_{l=1}^{n} e^{s_l}}}_{\text{poids : } \alpha_j \geq 0,\ \sum_j \alpha_j = 1} \qquad \underbrace{c = \sum_{j=1}^{n} \alpha_j \, v_j}_{\text{contexte}}$$

L'attention est une **recherche douce et différentiable** dans une mémoire : au lieu de récupérer une seule valeur (comme un dictionnaire), on récupère une moyenne pondérée de toutes les valeurs, pondérée par la similarité requête/clé.

Auto-attention (*self-attention*)

Dans l'auto-attention, requêtes, clés et valeurs proviennent **de la même séquence** : chaque token interroge tous les autres tokens (y compris lui-même) pour construire une nouvelle représentation enrichie de son contexte. Dans « Le chat dort car *il* est fatigué », la représentation de « il » va puiser principalement dans celle de « chat » : la coréférence est résolue en une seule couche.

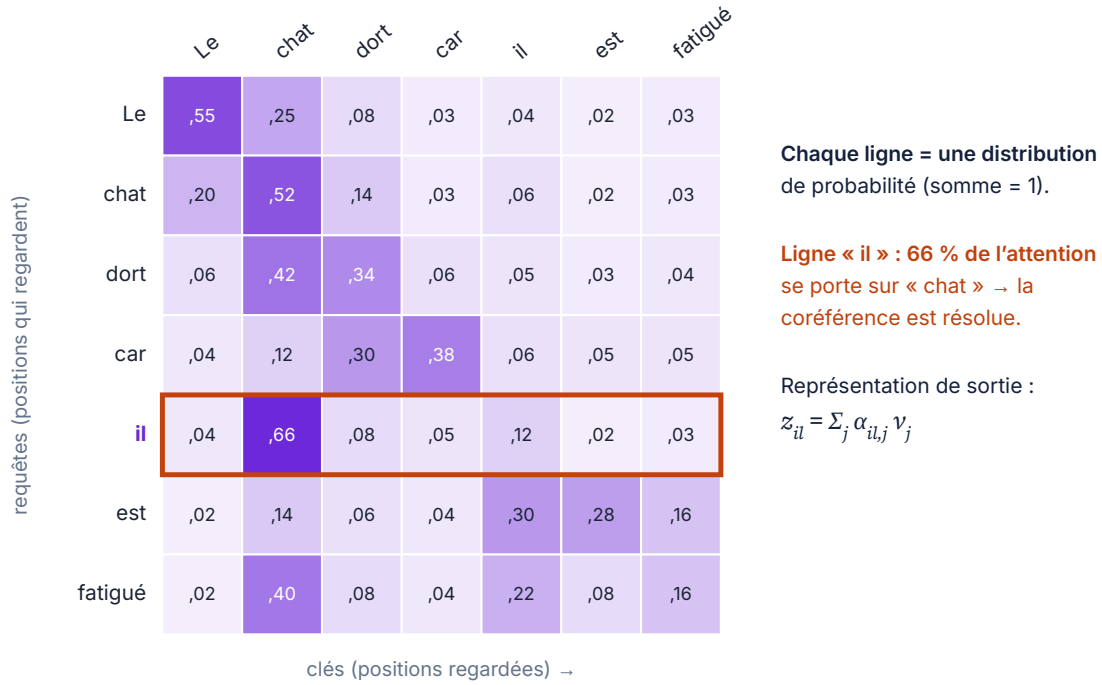


Figure 11.2 — Matrice d'auto-attention (valeurs illustratives). La ligne i donne la répartition de l'attention du token i sur tous les tokens.

Propriété	RNN	Auto-attention
Chemin entre deux positions distantes de n	$O(n)$ étapes : information diluée	$O(1)$: connexion directe
Parallélisation sur la séquence	Impossible (séquentiel)	Totale (produits matriciels sur GPU)
Coût par couche	$O(n\,d^2)$	$O(n^2\,d)$: quadratique en longueur
Notion d'ordre	Intrinsèque	Absente : il faut l'ajouter (11.5)

11.3 Query, key et value

Chaque token joue trois rôles, obtenus par trois projections linéaires apprises de son embedding $x_i \in \mathbb{R}^d$:

- **Query** (requête) $q_i = x_i W^Q$: « qu'est-ce que je cherche ? »
- **Key** (clé) $k_j = x_j W^K$: « qu'est-ce que je contiens, comment me trouver ? »
- **Value** (valeur) $v_j = x_j W^V$: « quelle information je transmets si l'on me sélectionne ? »

Séparer ces rôles donne beaucoup de souplesse : deux tokens peuvent être très pertinents l'un pour l'autre (fort $q_i \cdot k_j$) sans être similaires, et l'information transmise (v_j) peut différer de celle qui sert à la correspondance (k_j).

ATTENTION PAR PRODUIT SCALAIRE NORMALISÉ (SCALED DOT-PRODUCT ATTENTION)

Pour une séquence $X \in \mathbb{R}^{n \times d}$: $Q = XW^Q$, $K = XW^K \in \mathbb{R}^{n \times d_k}$, $V = XW^V \in \mathbb{R}^{n \times d_v}$.

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V}$$

$QK^\top \in \mathbb{R}^{n \times n}$ contient tous les scores $q_i \cdot k_j$; le softmax s'applique **ligne par ligne** ; M est un masque optionnel (0 ou $-\infty$).

Pourquoi diviser par $\sqrt{d_k}$? Si les composantes de q et k sont indépendantes, de moyenne 0 et de variance 1, alors $q \cdot k = \sum_{l=1}^{d_k} q_l k_l$ a une variance égale à d_k. Pour $d_k = 64$, les scores ont un écart-type de 8 : le softmax sature (un poids proche de 1, les autres proches de 0) et ses gradients s'annulent. La division ramène la variance à 1.

Masques : le *masque de remplissage* met à $-\infty$ les scores vers les tokens `[PAD]` ; le *masque causal* met à $-\infty$ les scores vers les positions futures ($j > i$), ce qui donne un poids nul après le softmax ($e^{-\infty} = 0$). Il est indispensable pour la génération de texte (11.7).

EXEMPLE CHIFFRÉ : 3 TOKENS, DIMENSION 2

Clés $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$; valeurs $v_1 = (1, 0)$, $v_2 = (0, 2)$, $v_3 = (1, 1)$; requête du token 3 : $q_3 = (1, 1)$.

$$s = \tfrac{1}{\sqrt{2}}\,(q_3 \cdot k_1,\ q_3 \cdot k_2,\ q_3 \cdot k_3) = (0{,}71;\ 0{,}71;\ 1{,}41) \xrightarrow{\text{softmax}} \alpha = (0{,}25;\ 0{,}25;\ 0{,}50)$$

$$z_3 = 0{,}25\,(1, 0) + 0{,}25\,(0, 2) + 0{,}50\,(1, 1) = (0{,}75;\ 1{,}00)$$

Le token 3 s'intéresse surtout à lui-même (clé la plus alignée), mais intègre aussi l'information des deux autres.

```python
import math, torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K : (N, n, d_k)   V : (N, n, d_v)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))      # (N, n, n)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))      # positions interdites
    weights = scores.softmax(dim=-1)                               # chaque ligne somme à 1
    return weights @ V, weights                                    # (N, n, d_v)

n = 5
causal = torch.tril(torch.ones(n, n))                              # masque causal triangulaire
inférieur
# En pratique : F.scaled_dot_product_attention(Q, K, V, is_causal=True)  (implémentation optimisée)
```

11.4 Attention à plusieurs têtes

Une seule attention produit une seule distribution de poids par token : elle ne peut capturer qu'un type de relation à la fois. L'**attention multi-têtes** exécute h attentions en parallèle, chacune avec ses propres projections, dans des sous-espaces de dimension réduite $d_k = d/h$. Chaque tête peut se spécialiser : relations syntaxiques (sujet-verbe), coréférence, position voisine, ponctuation…

FORMULES

$$\text{head}_i = \text{Attention}\left(XW_i^Q,\ XW_i^K,\ XW_i^V\right), \qquad W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O, \qquad W^O \in \mathbb{R}^{hd_k \times d}$$

Avec $d_k = d/h$, le coût total est le même qu'une seule tête de dimension d. Paramètres : $4d^2$ (plus les biais). BERT-base : $d = 768$, $h = 12$, $d_k = 64$.

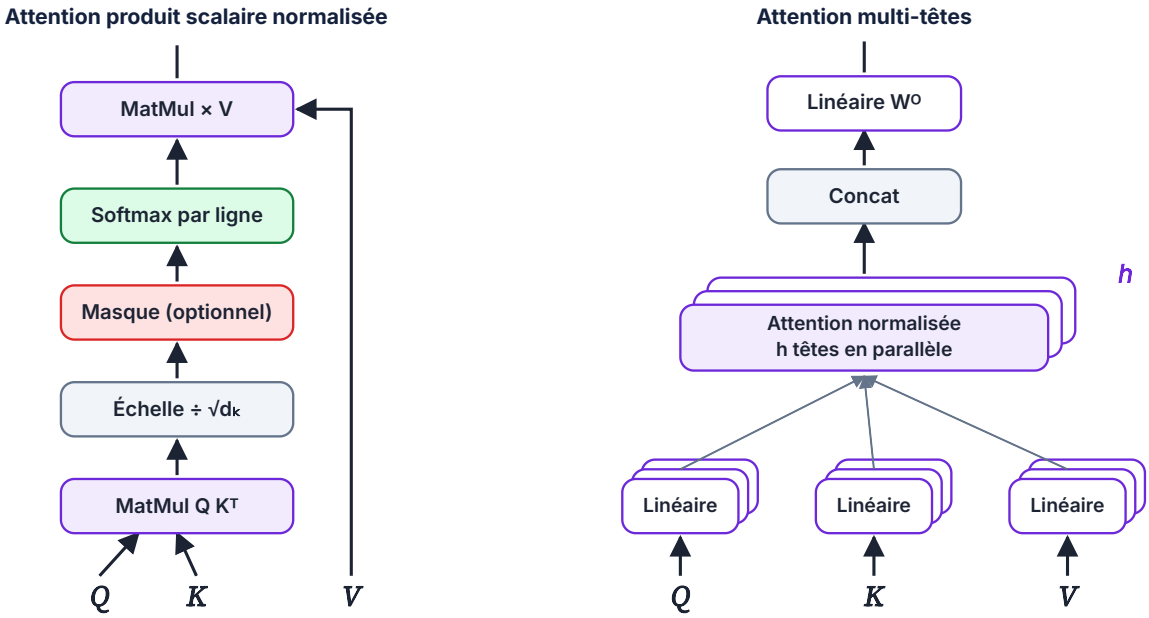


Figure 11.3 — À gauche : calcul d'une tête d'attention. À droite : h têtes en parallèle, concaténées puis projetées par W^O.

```python
mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, dropout=0.1, batch_first=True)
x = torch.randn(4, 20, 512)                                    # (N, n, d)
out, attn = mha(x, x, x, key_padding_mask=pad_mask)            # auto-attention : Q = K = V = x
print(out.shape, attn.shape)                                   # (4, 20, 512)  (4, 20, 20) moyenne des têtes
```

11.5 Information de position

L'auto-attention est **équivariante par permutation** : si l'on mélange les tokens d'entrée, les sorties sont mélangées de la même façon, sans autre changement. Sans information supplémentaire, « le chien mord l'homme » et « l'homme mord le chien » seraient indiscernables. On injecte donc la **position** de chaque token.

Encodage sinusoïdal (Transformer original)

FORMULE

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Le vecteur de position est **ajouté** à l'embedding : $x_{pos} = e_{token} + PE_{pos}$. Chaque paire de dimensions est une sinusoïde de fréquence différente (longueurs d'onde de 2π à $10000 \cdot 2\pi$) : un « compteur » à plusieurs échelles. Pour tout décalage k, PE_{pos+k} est une **fonction linéaire** (une rotation) de PE_{pos}, ce qui facilite l'apprentissage des positions relatives. Aucun paramètre appris.

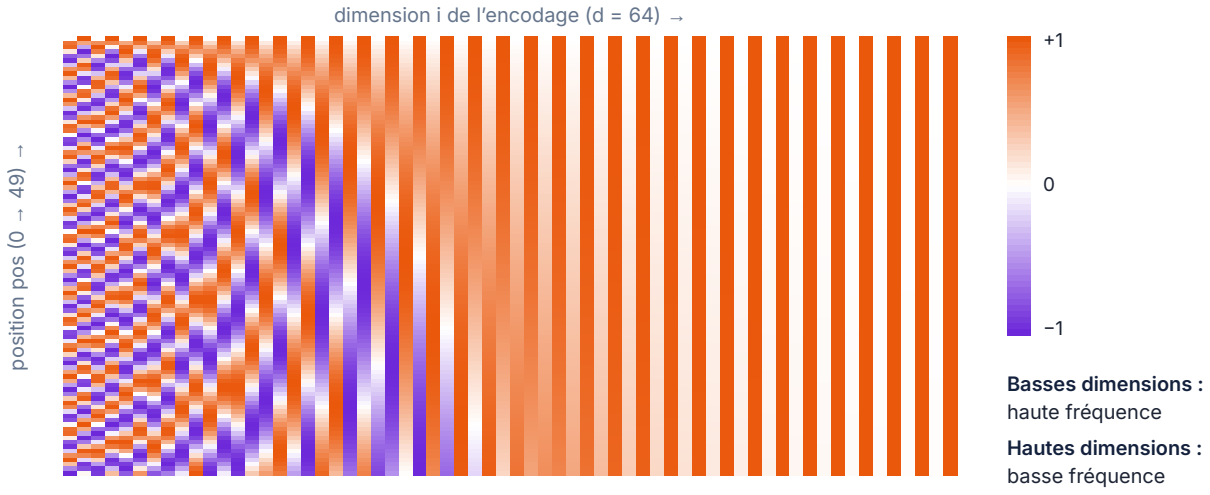


Figure 11.4 — Encodage positionnel sinusoïdal : chaque ligne est le vecteur ajouté à la position pos.

Autres approches

- **Positions apprises** (BERT, GPT-2) : une seconde table d'embedding $P \in \mathbb{R}^{L_{max} \times d}$. Simple, mais impossible d'aller au-delà de L_{max} (512 pour BERT).
- **RoPE** (*Rotary Position Embedding* ; Llama, Mistral, Qwen) : on fait **tourner** chaque paire de composantes de q et k d'un angle proportionnel à la position, $\theta_{m,i} = m \cdot 10000^{-2i/d}$. Le produit scalaire $\langle R_m q, R_n k \rangle$ ne dépend alors que de la **distance relative** $m - n$. C'est la méthode dominante des LLM actuels.

- **ALiBi** : pas d'encodage ; on ajoute aux scores un biais négatif proportionnel à la distance, $-m_h\,|i-j|$, ce qui favorise l'extrapolation à des séquences plus longues.

11.6 Architecture générale des Transformers

Un Transformer empile N **blocs identiques** (6 dans l'article original, 12 pour BERT-base, 32 à 128 pour les LLM). Chaque bloc combine deux sous-couches, chacune entourée d'une **connexion résiduelle** et d'une **normalisation de couche** :

1. **Attention multi-têtes** : *mélange l'information entre positions* (communication entre tokens).
2. **Réseau feed-forward position par position** (FFN) : un MLP à deux couches appliqué *indépendamment* à chaque token (traitement local, « mémoire » des connaissances).

BLOC TRANSFORMER (VARIANTE PRE-LN, STANDARD ACTUEL)

$$X' = X + \mathrm{MultiHead}\left(\mathrm{LN}(X)\right)$$
$$X'' = X' + \mathrm{FFN}\left(\mathrm{LN}(X')\right), \qquad \mathrm{FFN}(x) = W_2\ \mathrm{GELU}(W_1 x + b_1) + b_2$$

$W_1 \in \mathbb{R}^{d_{ff}\times d}$ avec habituellement $d_{ff} = 4d$. Normalisation de couche (par token, sur les d composantes) :

$$\mathrm{LN}(x) = \gamma \odot \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_l x_l, \quad \sigma^2 = \frac{1}{d}\sum_l (x_l-\mu)^2$$

L'article original place la normalisation après l'addition (Post-LN : $\mathrm{LN}(X + \mathrm{Sublayer}(X))$) ; Pre-LN est plus stable pour les réseaux profonds.

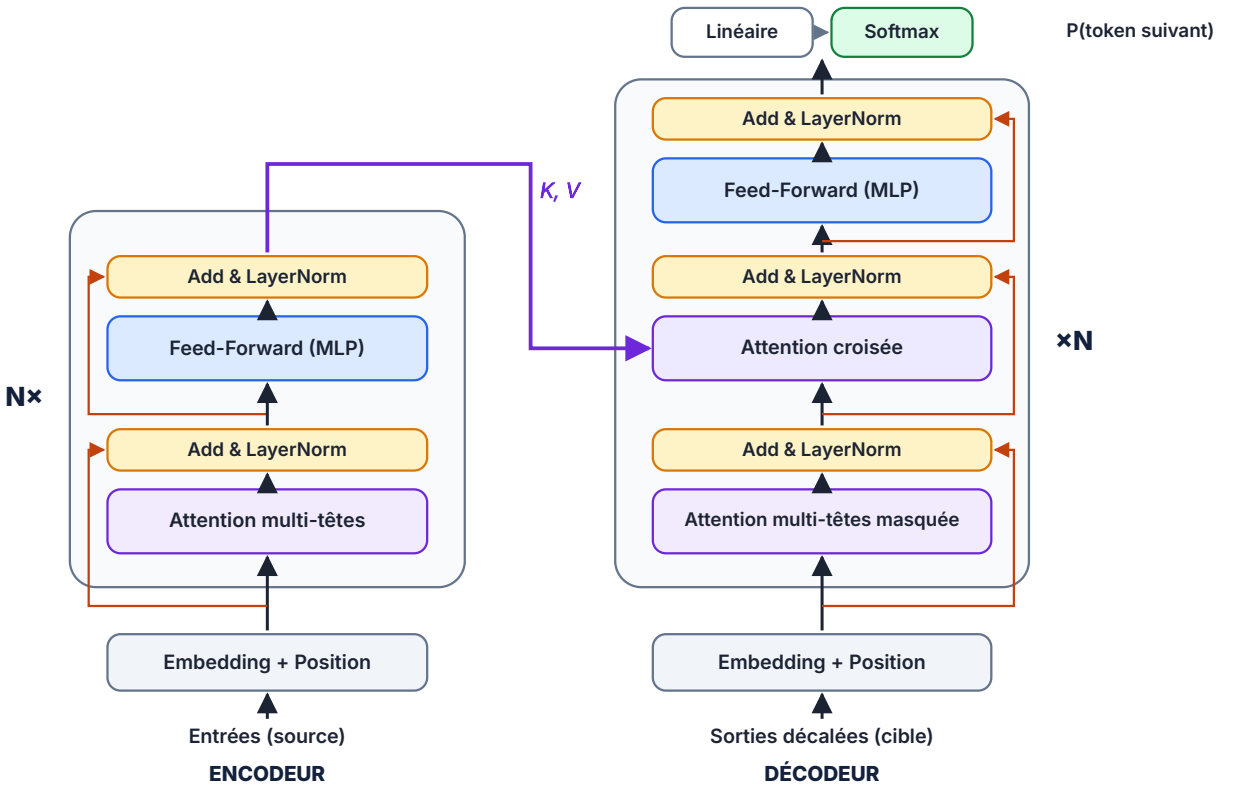


Figure 11.5 — Architecture encodeur-décodeur du Transformer original (Post-LN). Les flèches orange sont les connexions résiduelles.

Rôle de chaque composant

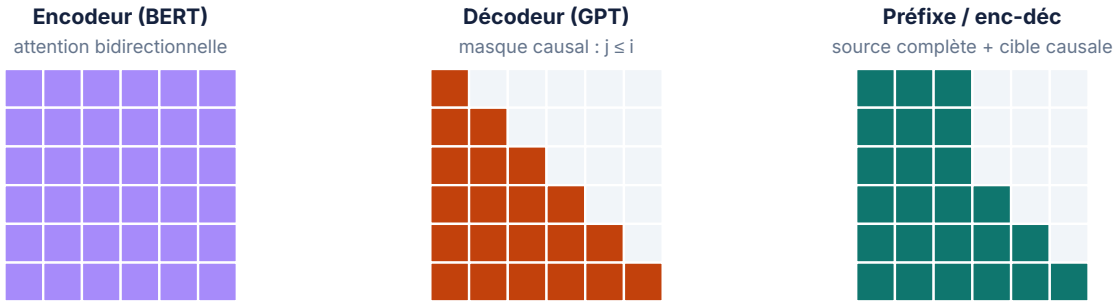
- **Connexions résiduelles** : comme dans ResNet (9.4), elles assurent la circulation du gradient dans des piles de dizaines de blocs ; on peut voir le réseau comme un « flux résiduel » que chaque sous-couche lit et enrichit.
- **LayerNorm** : stabilise l'échelle des activations, indépendamment de la taille du lot (contrairement à BatchNorm), ce qui convient aux séquences de longueur variable. Variante RMSNorm dans les LLM récents.
- **Dropout** sur les poids d'attention et les sorties des sous-couches ; **tête de sortie** : couche linéaire $d \to |V|$ + softmax (souvent liée à la matrice d'embedding $E^\top$).

ORDRE DE GRANDEUR DES PARAMÈTRES

Par bloc : attention $4d^2$ + FFN $2 \cdot d \cdot 4d = 8d^2$ $\approx 12d^2$. BERT-base : $12 \times 12 \times 768^2 \approx 85$ M + embeddings ≈ 24 M $\approx$ **110 M**. Le coût en mémoire de l'attention croît en $O(n^2)$ avec la longueur du contexte, d'où les optimisations comme **FlashAttention** (calcul par tuiles, sans stocker la matrice $n \times n$).

11.7 Encodeurs et décodeurs

Le Transformer original est un **encodeur-décodeur** conçu pour la traduction. Ses deux moitiés ont donné naissance à trois familles de modèles, qui se distinguent par le **masque d'attention** et l'**objectif de préentraînement**.



Case colorée = la position i (ligne) peut regarder la position j (colonne) ; case grise = score mis à −∞.

Figure 11.6 — Masques d'attention des trois familles (6 tokens).

Encodeur seul : comprendre (BERT, RoBERTa, CamemBERT, DeBERTa)

Attention **bidirectionnelle** : chaque token voit toute la phrase. Préentraînement par **modèle de langue masqué** (MLM) : on masque 15 % des tokens et le modèle doit les retrouver à partir du contexte gauche *et* droit, $\mathcal{L}_{MLM} = -\sum_{i \in \mathcal{M}} \log p(x_i \mid x_{\setminus \mathcal{M}})$. Produit d'excellentes représentations contextuelles pour la classification, l'extraction d'entités nommées, la recherche sémantique (embeddings de phrases), les questions-réponses extractives. Ne génère pas de texte.

Décodeur seul : générer (GPT, Llama, Mistral, Claude)

Attention **causale** : le token t ne voit que les tokens $\leq t$. Préentraînement par **prédiction du token suivant** (modèle de langue autorégressif) :

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad \mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

Grâce au masque, les T prédictions d'une séquence sont entraînées **en parallèle**. En génération, on produit un token, on l'ajoute à l'entrée et on recommence ; le **cache KV** mémorise les clés et valeurs déjà calculées pour éviter de les recalculer. Stratégies de décodage :

- **Glouton** (token le plus probable) ou **beam search** (garde les B meilleures séquences partielles) : déterministes, adaptés à la traduction ;
- **Échantillonnage avec température** $p_i \propto \exp(z_i/\tau)$: $\tau < 1$ rend la génération plus conservatrice, $\tau > 1$ plus créative ;
- **top-k** (échantillonner parmi les k tokens les plus probables) et **top-p** / *nucleus* (plus petit ensemble de tokens dont la probabilité cumulée atteint p).

Encodeur-décodeur : transformer une séquence en une autre (T5, BART, mBART, Whisper)

L'encodeur lit la source avec une attention bidirectionnelle. Chaque bloc du décodeur ajoute une couche d'**attention croisée** : les **requêtes** viennent du décodeur, les **clés et valeurs** viennent de la sortie de l'encodeur, $\text{Attention}(Q_{dec}, K_{enc}, V_{enc})$. Idéal pour la traduction, le résumé, la reconnaissance vocale (Whisper : audio → texte).

Famille	Attention	Préentraînement	Exemples	Tâches typiques
Encodeur seul	Bidirectionnelle	Tokens masqués (MLM)	BERT, RoBERTa, CamemBERT	Classification, NER, embeddings, recherche
Décodeur seul	Causale	Token suivant	GPT, Llama, Mistral	Génération, chat, code, raisonnement
Encodeur-décodeur	Bidir. + causale + croisée	Débruitage de séquence	T5, BART, Whisper	Traduction, résumé, parole → texte

AU-DELÀ DU TEXTE

Le même bloc s'applique à toute donnée découpée en tokens : **ViT** (patchs d'image 16×16 projetés linéairement), audio (trames de spectrogramme), séries temporelles (PatchTST), multimodal (CLIP aligne images et textes). Les Transformers sont aujourd'hui l'architecture universelle du deep learning.

11.8 Utilisation et adaptation de modèles préentraînés avec Hugging Face

Préentraîner un Transformer coûte de quelques milliers à plusieurs millions d'euros de calcul. L'écosystème **Hugging Face** donne accès à des centaines de milliers de modèles préentraînés et standardise leur utilisation.

Bibliothèque	Rôle
Hub (huggingface.co)	Hébergement de modèles, jeux de données et démos, avec *model cards* (usage, données, limites, licence).
`transformers`	Architectures, tokeniseurs, `pipeline`, classes `Auto*`, `Trainer`.
`datasets`	Chargement et prétraitement efficace (mémoire mappée, `map` par lots).
`tokenizers`	Tokeniseurs rapides (Rust).
`peft`	Fine-tuning efficace en paramètres (LoRA, adapters, prompt tuning).
`evaluate`, `accelerate`	Métriques ; entraînement multi-GPU et précision mixte.

1. Utilisation directe : `pipeline`

```python
from transformers import pipeline

clf = pipeline("text-classification", model="cmarkea/distilcamembert-base-sentiment")
print(clf("Ce cours sur les Transformers est vraiment clair !"))
# [{'label': '5 stars', 'score': 0.8...}]

gen = pipeline("text-generation", model="gpt2")
ner = pipeline("token-classification", model="Jean-Baptiste/camembert-ner",
aggregation_strategy="simple")
```

Le pipeline enchaîne tokenisation → modèle → post-traitement (softmax, étiquettes). Pour plus de contrôle, on utilise les classes `AutoTokenizer` et `AutoModel*` : `AutoModel` donne les états cachés (embeddings contextuels), `AutoModelForSequenceClassification` ajoute une tête de classification, `AutoModelForCausalLM` une tête de génération.

2. Stratégies d'adaptation

Stratégie	Principe	Quand
Zero-shot / prompting	Aucun entraînement : on décrit la tâche (LLM) ou on utilise un modèle NLI zero-shot.	Prototype, pas de données étiquetées.
Extraction de caractéristiques	Modèle gelé ; embedding du token [CLS] ou moyenne des tokens → classifieur simple.	Peu de données, peu de calcul.
Fine-tuning complet	Tous les poids sont mis à jour, lr $\approx 2 \cdot 10^{-5}$, 2 à 5 époques.	Quelques milliers d'exemples, modèle < 1 Md de paramètres.
PEFT (LoRA, QLoRA)	Modèle gelé + petites matrices entraînables (≈ 0,1–1 % des paramètres).	Grands modèles, mémoire GPU limitée.

3. Fine-tuning d'un encodeur pour la classification

```python
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer, DataCollatorWithPadding)
import numpy as np, evaluate

ckpt = "almanach/camembert-base"
ds = load_dataset("csv", data_files={"train": "train.csv", "validation": "val.csv"})  # colonnes text, label
tok = AutoTokenizer.from_pretrained(ckpt)
ds = ds.map(lambda b: tok(b["text"], truncation=True, max_length=256), batched=True)

model = AutoModelForSequenceClassification.from_pretrained(ckpt, num_labels=3)  # nouvelle tête aléatoire
f1 = evaluate.load("f1")
def compute_metrics(p):
    return f1.compute(predictions=np.argmax(p.predictions, -1), references=p.label_ids, average="macro")

args = TrainingArguments("out", learning_rate=2e-5, per_device_train_batch_size=16,
                         num_train_epochs=3, weight_decay=0.01, warmup_ratio=0.1,
                         eval_strategy="epoch", save_strategy="epoch",
                         load_best_model_at_end=True, metric_for_best_model="f1", fp16=True)
trainer = Trainer(model=model, args=args, train_dataset=ds["train"], eval_dataset=ds["validation"],
                  data_collator=DataCollatorWithPadding(tok), compute_metrics=compute_metrics)
trainer.train()
trainer.save_model("mon-modele"); tok.save_pretrained("mon-modele")
```

4. LoRA : adaptation à faible rang

Hypothèse (Hu et al., 2021) : la mise à jour nécessaire pour adapter un modèle a un **rang intrinsèque faible**. Au lieu d'apprendre $\Delta W \in \mathbb{R}^{d\times d}$, on apprend deux petites matrices :

FORMULE LORA

$$h = W_0\,x + \frac{\alpha}{r}\,B A\,x, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d$$

W_0 est gelé ; B est initialisé à zéro, donc le modèle de départ est inchangé. Paramètres entraînables : $2dr$ au lieu de d^2 ; pour $d = 4096$ et $r = 8$: 65 536 contre 16,8 M, soit **0,4 %**. Après l'entraînement, on peut fusionner $W = W_0 + \frac{\alpha}{r}BA$: aucun surcoût à l'inférence. **QLoRA** combine LoRA avec un modèle de base quantifié en 4 bits.

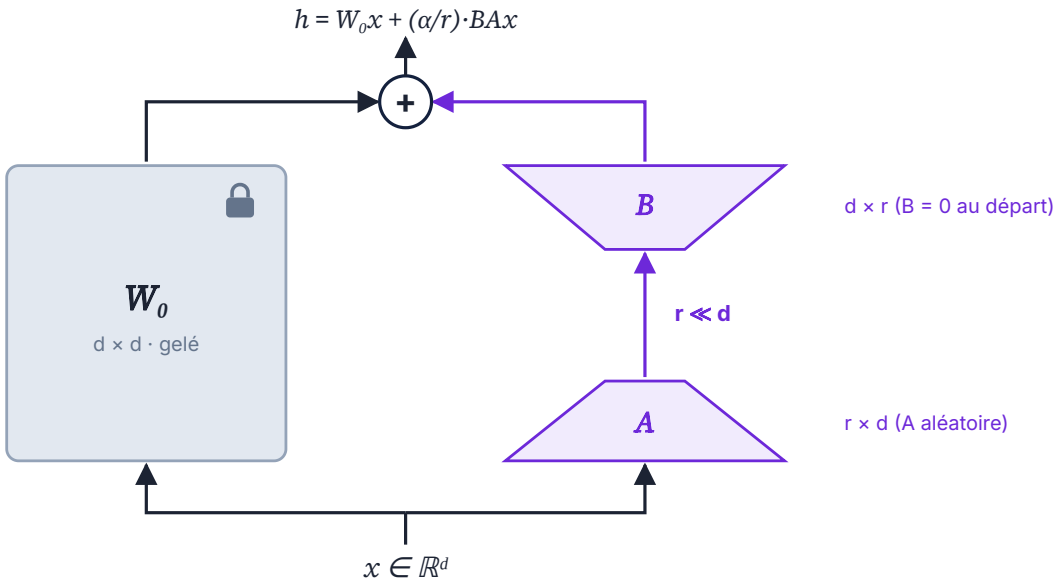


Figure 11.7 — LoRA : une dérivation de faible rang (B·A) est ajoutée à la matrice gelée W_0.

```python
from peft import LoraConfig, get_peft_model

config = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.1,
                    target_modules=["query", "value"],      # projections W^Q et W^V de l'attention
                    task_type="SEQ_CLS")
model = get_peft_model(model, config)
model.print_trainable_parameters()   # trainable params : ~0,9 M || all params : ~111 M (tête comprise)
# → entraînement avec le même Trainer que ci-dessus (lr plus élevé possible : 1e-4 à 5e-4)
```

BONNES PRATIQUES

- Utiliser **le tokeniseur du même checkpoint** que le modèle, et contrôler `max_length` / `truncation` (limite de contexte du modèle).
- Choisir un modèle adapté à la **langue** et au domaine (CamemBERT, FlauBERT pour le français ; modèles multilingues sinon).
- Lire la *model card* : données d'entraînement, biais connus, **licence** (usage commercial ou non).
- Taux d'apprentissage faible avec *warmup* ; évaluer sur un jeu de validation et comparer à une référence simple (TF-IDF + régression logistique).
- Fixer les graines (`transformers.set_seed`) et versionner modèle, tokeniseur et données pour la reproductibilité.

À RETENIR — PARTIE 11

- Le texte est découpé en sous-mots (BPE, WordPiece, Unigram), convertis en identifiants puis en embeddings appris $E \in \mathbb{R}^{|V| \times d}$.
- L'attention calcule une moyenne pondérée des valeurs, avec des poids donnés par la similarité requête/clé ; en auto-attention, Q, K et V viennent de la même séquence.
- $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$; la division par $\sqrt{d_k}$ évite la saturation du softmax ; les masques gèrent padding et causalité.
- Plusieurs têtes captent plusieurs types de relations ; l'information de position (sinusoïdale, apprise, RoPE) est indispensable.
- Bloc = attention + FFN, chacun avec résiduelle et LayerNorm ; $\approx 12d^2$ paramètres par bloc.
- Encodeur (BERT) pour comprendre, décodeur (GPT) pour générer, encodeur-décodeur (T5) pour transformer.
- Hugging Face : `pipeline` pour utiliser, `Trainer` pour fine-tuner, `peft`/LoRA pour adapter les grands modèles à moindre coût.

ANNEXE

Formulaire de synthèse et références

Les formules essentielles des parties 9 à 11 réunies sur une page, suivies des articles fondateurs à consulter pour approfondir.

A Formulaire de synthèse

Notion	Formule
Convolution 2D	$Y_k[i,j] = b_k + \sum_{c,u,v} W_c^{(k)}[u,v]\, X_c[iS+u,\, jS+v]$
Taille de sortie	$H_{out} = \lfloor (H_{in} + 2P - K)/S \rfloor + 1$
Paramètres Conv2d	$C_{out}\, C_{in}\, K^2 + C_{out}$
Champ réceptif	$r_l = r_{l-1} + (K_l - 1)\prod_{i<l} S_i$
Bloc résiduel	$y = \mathrm{ReLU}(F(x) + x)$
Fenêtres glissantes	$N = T - L - H + 1$ exemples de forme $(L, F) \to (H)$
RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$
BPTT	$\partial h_t / \partial h_k = \prod_{i=k+1}^{t} \mathrm{diag}(1 - h_i^2)\, W_{hh}$
Écrêtage du gradient	$g \leftarrow \tau\, g / \lVert g \rVert$ si $\lVert g \rVert > \tau$
LSTM (mémoire)	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$
GRU	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
MAE / RMSE	$\frac{1}{n}\sum \lvert y - \hat{y} \rvert \qquad \sqrt{\frac{1}{n}\sum (y - \hat{y})^2}$
Embedding	$e_i = E[i, :], \quad E \in \mathbb{R}^{\lvert V \rvert \times d}$
Attention	$\mathrm{softmax}\left(QK^\top / \sqrt{d_k} + M\right) V$
Multi-têtes	$\mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O, \quad d_k = d/h$
Position sinusoïdale	$PE_{pos,2i} = \sin(pos/10000^{2i/d}), \quad PE_{pos,2i+1} = \cos(pos/10000^{2i/d})$
Bloc Transformer	$X' = X + \mathrm{MHA}(\mathrm{LN}(X)), \quad X'' = X' + \mathrm{FFN}(\mathrm{LN}(X'))$
Modèle de langue causal	$p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$
Température	$p_i \propto \exp(z_i/\tau)$
LoRA	$h = W_0 x + \frac{\alpha}{r} B A x, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times d}$

B Références essentielles

1. LeCun, Bottou, Bengio, Haffner (1998). *Gradient-Based Learning Applied to Document Recognition*. Proc. IEEE. — LeNet-5.

2. Krizhevsky, Sutskever, Hinton (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. NeurIPS. — AlexNet.
3. Simonyan, Zisserman (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR. — VGG.
4. He, Zhang, Ren, Sun (2016). *Deep Residual Learning for Image Recognition*. CVPR. — ResNet.
5. Yosinski, Clune, Bengio, Lipson (2014). *How transferable are features in deep neural networks?* NeurIPS.
6. Hochreiter, Schmidhuber (1997). *Long Short-Term Memory*. Neural Computation.
7. Cho et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP. — GRU.
8. Pascanu, Mikolov, Bengio (2013). *On the difficulty of training recurrent neural networks*. ICML.
9. Bahdanau, Cho, Bengio (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR.
10. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS. — Transformer.
11. Devlin, Chang, Lee, Toutanova (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
12. Radford et al. (2019). *Language Models are Unsupervised Multitask Learners*. — GPT-2.
13. Raffel et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR. — T5.
14. Dosovitskiy et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR. — ViT.
15. Su et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. — RoPE.
16. Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR.
17. Documentation : *pytorch.org/docs*, *pytorch.org/vision*, *huggingface.co/docs/transformers*, *huggingface.co/learn* (cours NLP gratuit).